

Geodatenverarbeitung mit OpenSource Komponenten 2025

Kurs ETH Zürich - Planung 2025

Version: 1.03

Datum: 21.05.2025

Autor: Prof Hans-Jörg Stark

Hilfreiche Referenzen:

<https://pcjericks.github.io/py-gdalogr-cookbook/layers.html>

<https://pcjericks.github.io/py-gdalogr-cookbook/geometry.html>

Kursaufbau

Der Kurs ist wie folgt aufgebaut:

Teil I - Analyse von Geodaten

- Analyse von Vektordaten (Format, Attribute, Geometrietyp etc.)
- Analyse von Rasterdaten (Grösse, Dimension etc.)

Teil II - Spezifische Aufgaben

Weitere Operationen an räumlichen Daten:

- Filtern von Daten
- Umprojektion
- Analyse der Räumlichen Datenstrukturen
- Extraktion bestimmter impliziter Kenngrössen (bspw Zentroid)
- Extraktion radiometrischer Werte aufgrund gegebener Koordinaten
- Extraktion spezifischer Informationen aus Höhenmodelldaten (Schumierung, Neigung, Exposition, Ausrichtung, Höhenlinien)

Teil III - Erstellung von Geodaten

- Erstellen von Vektordaten (Punkte, Linien, Flächen)
- Erstellen von Rasterdaten (Rasterdateien)

Teil IV - Spezifische Aufgaben:

- Extraktion MBR von Flächendaten
- Arbeiten mit räumlichen Datenbanken
- Arbeiten mit OGC Webdiensten

Teil V - Weitere Bibliotheken:

- Shapely
- Fiona

- Folium
- Leafmap
- DuckDB

[Top](#)

Teil I - Analyse von Geodaten

Im Folgenden werden erste Analysen mit dem frei verfügbaren Datensatz der Gemeinden aus dem Kanton Solothurn gemacht. Diese befinden sich im Datenverzeichnis `Data` unter dem Namen `Gemeinden_Solothurn.shp`

Werden diese visualisiert, präsentiert sich folgende Ansicht:



Als erstes müssen für die Verarbeitung der Daten mit Python die notwendigen Module und Komponenten geladen werden, wie das mit Python üblich ist. Dies erfolgt mit dem `import` Befehl:

```
from osgeo import ogr, gdal
```

```
In [ ]: import os
        from osgeo import ogr, gdal
        import sys
```

Mit der Methode `VersionInfo()` kann die Version der aktuell installierten gdal Version abgefragt werden.

```
In [ ]: version_num = int(gdal.VersionInfo('VERSION_NUM'))
        if version_num < 1100000:
            sys.exit('ERROR: Python bindings of GDAL 1.10 or later required')

        print(f"Version: {version_num}")
```

Fehlerhandling:

Eine generelle Vorbemerkung zum Code-Schreiben:

In den allermeisten Fällen funktioniert der geschriebene Quellcode nicht immer zu 100%. Es tauchen Fehler auf durch die Wahl falscher oder ungenügender Parameter, Schreibfehler, Syntaxfehler etc. Daher ist ein gutes Fehlerhandling hilfreich und sinnvoll. Dies kann beispielsweise erfolgen durch den Einsatz von `try` und `except` wie das folgende Beispiel zeigt:

```
try:
    a = 1/0
except Exception as e:
    print(e)
```

Für die Analyse von Geodaten sind folgende Informationen relevant:

- Pfad zu den und Name der Geodaten, die zu analysieren sind
- Treiber, mit welchem die Geodaten in ihrem nativen Format geladen und analysiert werden können

Dazu helfen die Befehle `GetDriverByName()` und von dem initiierten Treiberobjekt schliesslich die Methode `Open()`.

```
In [ ]: # Initiieren des korrekten Treibers und Laden der Geodaten

drv = ogr.GetDriverByName("Esri Shapefile")
path2ds = os.path.join("../Data/", "Gemeinden_Solothurn.shp")
#path2ds = r"U:\IKG\Modul4_2025\Data"
print(path2ds)

datasource = drv.Open(path2ds)
print(datasource)
```

Beschreibung Esri Shapefile

Ein Whitepaper zum Esri Shapefile ist zu finden unter: [Esri Shapefile](#)

Ebenso empfohlen der Wikipedia Artikel: [Esri Shapefile](#)

Beschreibung OGR Vektor Modell

Das Vektormodell von OGR ist wie folgt definiert:  No description has been provided for this image

```
In [ ]: from osgeo import ogr

# Pfad zur Shapefile
shapefile_path = r"U:\IKG\Modul4_2025\Data"

# Shapefile öffnen
driver = ogr.GetDriverByName("ESRI Shapefile")
data_source = driver.Open(shapefile_path, 0) # 0 = read-only, 1 = writeable

# Layer extrahieren
layer = data_source.GetLayer()

# Erste Features anzeigen
for i, feature in enumerate(layer):
    print(f"Feature {i}:")
    print("  ID:", feature.GetFID())
    print("  Name:", feature.GetField("Name") if feature.GetField("Name") else "kein")
    #print("  Geometrie:", feature.GetGeometryRef().ExportToWkt())
    if i >= 4: # nur die ersten 5 anzeigen
        break
```

```
In [ ]: from pathlib import Path

path2ds = r"C:\Users\starkhan\Documents\geoproc\Data\Gemeinden_Solothurn.shp"

file_path = Path(path2ds)
if file_path.is_file():
    print("File exists!")
else:
    print("File does not exist.")
```

Layeridentifikation

Wurden die Daten erfolgreich geladen, kann auf das Layerobjekt des Datensatzes mit Hilfe von `GetLayer()` zugegriffen werden. Dies ist notwendig, um schliesslich Zugriff zu den einzelnen Objekten und der Struktur des Datensatzes (bspw. Attributnamen etc.) zu erhalten.

```
In [ ]: # Definition des Layers der geladenen Shapedatei

layer = datasource.GetLayer(0)
print(layer)
```

Objektanzahl

Da nun die Daten geladen sind, kann beispielsweise die Anzahl der vorhandenen Einträge oder Objekte oder Records ermittelt werden. Dies geschieht mit dem Befehl `GetFeatureCount()`.

```
In [ ]: # Ermittlung der Anzahl Datensätze im Layer

ftrCnt = layer.GetFeatureCount()
print(f"Die Anzahl der Gemeinden im Kanton Solothurn beträgt {ftrCnt}.")
```

CLI-basierte Befehle

Nebst den Python-nativen Befehlen, die hier zur Anwendung kommen, bietet OSGEO auch CLI-basierte Befehle. So ist es möglich, mit dem Befehl `ogr2ogr` Geodaten von einem Format in ein anderes umzuwandeln und in diesem Kontext weitere Operationen durchzuführen (filtern, umprojizieren etc.).

Für detaillierte Informationen zu diesen Möglichkeiten sei auf die Seite <https://gdal.org/en/stable/programs/ogr2ogr.html> verwiesen.

Das folgende Beispiel transformiert die Gemeinden von Solothurn aus dem Format Esri Shape ins Format GeoJSON:

```
ogr2ogr -f GeoJSON Data/gemSo.geojson Data/Gemeinden_Solothurn.shp .
```

```
In [ ]: # Shape in GeoJSON umwandeln
gjsonFl = os.path.join("../Data/", "gemSo.geojson")
cmd = f'ogr2ogr -f GeoJSON {gjsonFl} {path2ds}'
print(cmd)

os.system(cmd)
```

```
In [ ]: import subprocess

def runsubprocess(runcommand):
    result = subprocess.run(
        runcommand,
        capture_output=True, text=True
    )

    print("STDOUT:", result.stdout)
    print("STDERR:", result.stderr)
    if result.returncode == 0:
        resMsg = "Ohne Fehler prozessiert"
    else:
        resMsg = "Ein Fehler trat auf!!"

    print(f"{resMsg} --> Exit-Code: {result.returncode}")
```

```
In [ ]: # Shape in GeoJSON umwandeln
gjsonFl = os.path.join("../Data/", "gemSo.geojson")

rcmd = ["ogr2ogr", "-f", "GeoJSON", gjsonFl, path2ds]

runsubprocess(rcmd)
```

Da die Gemeinden von Solothurn nun im GeoJSON Format vorliegen, können die bisher durchgeführten Analysen (Zählen der Objekte im Datensatz) analog auf diesen transformierten Datensatz ausgeführt werden:

```
drv = ogr.GetDriverByName("GeoJSON")
#path2ds = os.path.join("Data","gemSo.geojson")
datasource = drv.Open(gjsonFl)
layer = datasource.GetLayer(0)
ftrCnt = layer.GetFeatureCount()
print(f"Die Anzahl der Gemeinden im Kanton Solothurn beträgt {ftrCnt}.")
```

Attributinformationen

Als nächstes werden Attributinformationen aus dem vorhandenen Datensatz extrahiert. Um dies zu tun, muss zuerst die Definition des Layers ausgelesen werden und anschliessend können anhand dieser Information die Anzahl Attribute gezählt werden. Für diese Operationen sind die Funktionen `GetLayerDefn()` und `GetFieldCount()` notwendig. Letztere ist eine Methode der ersteren.

```
In [ ]: # Zugriff auf Attributinformationen

lyrDef = layer.GetLayerDefn()
fldCnt = lyrDef.GetFieldCount()
print(f"Die Anzahl der Attribute im Geodatensatz beträgt {fldCnt}.")
```

Attributnamen

Da die Anzahl der Attribute nun bekannt ist, kann deren Namen beispielsweise über ein Iteration über alle Attribute ermittelt werden. Auch hier kommt die Methode `GetLayerDefn()` zum Einsatz, indem deren Methode `GetFieldDefn(i)` auf das in der Iteration aktuelle Objekt (Attribut) angewendet wird:

```
lyrDef.GetFieldDefn(i).GetName()
```

```
In [ ]: for i in range(fldCnt):
        attName = lyrDef.GetFieldDefn(i).GetName()
        print(f"{i+1}. Attribut heisst {attName}")
```

Räumlicher Bezugsrahmen

Von Interesse ist auch der Räumliche Bezugsrahmen. Anhand dieser Information können im Anschluss die ausgelesenen Koordinatenwerte korrekt interpretiert und ggf weitere Operationen oder Berechnungen mit oder auf diesen durchgeführt werden. Das räumliche Referenzsystem hängt als Eigenschaft am Layer des Datensatzes:

```
layer.GetSpatialRef()
```

```
In [ ]: # SRS Extraktion

mySrs = layer.GetSpatialRef()
print(mySrs)
```

Räumliche Ausdehnung

Die räumliche Ausdehnung des Layers, dh die minimalen und maximalen Koordinaten im räumlichen Bezugsystem, welche von den Objekte eingenommen werden, werden über die Layer-Methode :

```
layer.GetExtent()
```

ermittelt.

```
In [ ]: # Ausdehnung der Geometrien, dh MBR ermitteln

myExtent = layer.GetExtent()
```

Damit können auch die vier Eckpunkte des MBR (minimum bounding rectangle) ermittelt werden:

```
In [ ]: print(f"Dies sind die 4 Eckpunkte des MBR: \n 1. Punkt: {myExtent[0]} / {myExtent[2]}")
```

Geometrietyp

Eine weitere wichtige Eigenschaft ist der Geometrietyp eines Datensatzes mit räumlichen Daten. Auch wenn heute gemischte Geometrietypen in gewissen Formaten erlaubt sind, ist es oft so, dass für einen Datensatz nur ein Geometrietyp zugelassen oder verwendet wird.

Der Geometrietyp hängt am Layer-Objekt und kann mit der Methode `GetGeomType()` abgefragt werden.

```
In [ ]: # Ermittle den Geometrietyp des Layers
geomTypeCode = layer.GetGeomType()
geomTypeName = ogr.GeometryTypeToName(geomTypeCode)

# Ausgabe
print(f"Der Geometrietyp der Ebene ist: {geomTypeName}")
```

Zugriff auf einzelne Objekte im Datensatz

Nachdem nun die wichtigsten Merkmale des Vektordatensatzes bekannt sind, sollen die einzelnen Datensätze ausgewertet werden. Dazu zählen

- Attributwerte
- Geometrieinformationen (Stützpunkte)

Dazu kann ein spezifisches Objekt des Layers direkt angesprochen werden mit dem Befehl `layer.GetFeature(<n>)`.

```
In [ ]: # Zugriff auf einzelne Features/Objekte/Datensätze

myFtr = layer.GetFeature(0)
print(myFtr)
```

[Top](#)

Teil II Spezifische Aufgaben

Attributfilter

Auf das Layerobjekt kann ein Attributfilter angewendet werden. Damit ist es möglich, eine Teilmenge des Datensatzes zu extrahieren und damit weiter zu arbeiten. Die Methode, die auf das Layerobjekt angewendet werden kann in diesem Fall lautet `SetAttributeFilter()`.

Um den Filter zurückzusetzen wird der Befehl `layer.SetAttributeFilter(None)` angewendet.

Aufgaben:

1. Wie lautet die BFS Nummer der Gemeinde Olten?
2. Wie viele Gemeinden beginnen mit dem Buchstaben "E" und wie lauten ihre Namen alphabetisch absteigend sortiert?

```
In [ ]: # Aufgabe 1
gemName='Olten'
layer.SetAttributeFilter(f"Name = '{gemName}'")

for feature in layer:
    print(f"Die BFS Nummer der Gemeinde {gemName} lautet: {feature.GetField('gem_bfs'")
```

```
In [ ]: # Aufgabe 2
gemNamen='E%'
layer.SetAttributeFilter(f"Name like '{gemNamen}'")
print(f"Die Anzahl Gemeinden im Kanton Solothurn, die mit dem Buchstaben 'E' beginnen")
filterList=[]
for feature in layer:
    filterList.append(feature.GetField("name"))

filterList.sort(reverse=True)
print(f"Die Liste der Gemeinden in absteigender alphabetischer Reihenfolge:\n{filterL
```

Analyse Geometriedaten

Es kann von Nutzen sein, die Geometriedaten eines Objekts zu untersuchen. Beispielsweise können so bei Multipolygonen die einzelnen Ringe analysiert werden oder es kann auf die einzelnen Stützpunkte zurückgegriffen werden etc. Dazu dient als Hilfe die folgende Methode, die in Python formuliert wurde:

```
def analyzeGeometry(geometry, indent=0):
    s = []
    s.append(" " * indent)
    s.append(geometry.GetGeometryName())
    if geometry.GetPointCount() > 0:
        s.append(" mit %d Stuetzpunkten" % geometry.GetPointCount())
    if geometry.GetGeometryCount() > 0:
        s.append(" enthaelt:")

    print ("".join(s))

    for i in range(geometry.GetGeometryCount()):
        print(i)
        analyzeGeometry(geometry.GetGeometryRef(i), indent+1)
```

Mittels der Methode `GetGeometryRef()`, die auf ein einzelnes Objekt/Feature angewendet wird, besteht Zugriff auf dessen Geometriedefinition und -daten. Wird diese Information in einem Objekt instanziiert, kann dies der eben präsentierten Funktion `analyzeGeometry()` übergeben werden.

```
In [ ]: def analyzeGeometry(geometry, indent=0):
    s = []
    s.append(" " * indent)
    s.append(geometry.GetGeometryName())
    if geometry.GetPointCount() > 0:
        s.append(" mit %d Stuetzpunkten" % geometry.GetPointCount())
    if geometry.GetGeometryCount() > 0:
        s.append(" enthaelt:")

    print ("".join(s))

    for i in range(geometry.GetGeometryCount()):
        print(i)
        analyzeGeometry(geometry.GetGeometryRef(i), indent+1)
```

```

curFtr = layer.GetFeature(7)
print(curFtr.GetField('name'))
myGeometry = curFtr.GetGeometryRef()
#print(myGeometry)

analyzeGeometry(myGeometry)

```

Aufgabe:

1. Ermittle die Gemeinde im Kanton Solothurn, die

- am meisten
- am wenigsten

Stützpunkte hat.

2. Ermittle die durchschnittliche Anzahl an Stützpunkten pro Gemeinde im Kanton Solothurn.

```

In [ ]: ftrDict = {}
for feature in layer:
    geometry = feature.GetGeometryRef()
    geomCnt = geometry.GetGeometryCount()
    for i in range(geomCnt):
        if geometry.GetGeometryRef(i).GetPointCount() > 0:
            pntCnt = geometry.GetGeometryRef(i).GetPointCount()
            ftrDict[feature.GetField('gmde_name')] = pntCnt
sortedFtrDict = sorted(ftrDict.items(), key=lambda x:x[1])

if len(ftrDict) > 0:
    avgPntCnt = int(sum(ftrDict.values()) / len(ftrDict))
    print(f"Die Gemeinde in Solothurn mit:\n    den wenigsten Stützpunkten:{sortedFtrD

```

Umprojektion von Geodaten

Immer wieder kann es von Nutzen sein, räumliche Daten von einem Referenzsystem in ein anderes umzuprojizieren. Dies kann als permanenter Prozess verstanden werden oder nur ad-hoc um die Koordinaten in einem anderen System anzuzeigen, aber dennoch die Originaldaten nicht zu verändern.

Das Modul `osgeo` stellt die Komponente `osr` zur Verfügung, mithilfe derer die Umprojektion erfolgen kann. Dabei liefert `osr` eine Methode `SpatialReference()` welche ein Referenzsystem annehmen oder ausgeben kann. Dazu wird eine Variable entsprechend instanziiert und anschliessend die Projektion zugewiesen. Dies wird sowohl für das ursprüngliche System als auch das Zielsystem gemacht und schliesslich können anhand dieser beiden Definitionen die Daten vom Ausgangs- ins Zielsystem projiziert werden. Damit dies erfolgreich geschieht, stellt `osr` eine weitere Methode `CoordinateTransformation()` zur Verfügung. Diese Methode nimmt als Parameter zuerst das Quell- und dann das Zielsystem auf und führt anschliessend als Parameter die Umprojektion auf eine Geometrie aus.

Dies bedeutet, dass eine `OGC`- Geometriedefinition wie `point` diese Umprojektionsdefinition mit der Methode `Transform()` aufnehmen und so die Umprojektion durchführen kann. Ist die

Umprojektion in ein neues Geometrieobjekt erfolgt, können die umprojizierten Koordinaten mit der Methode `ExportToWkt()` in eine menschenlesbare Form ausgegeben werden.

```
In [ ]: # Umprojektion von Geodaten

from osgeo import osr
source = osr.SpatialReference()
source.ImportFromEPSG(2056)
destination = osr.SpatialReference()
destination.ImportFromEPSG(4326)

transformationdef = osr.CoordinateTransformation(source, destination)
retransformationdef = osr.CoordinateTransformation(destination, source)

point = ogr.CreateGeometryFromWkt("POINT (2618579 1244235)")
point2 = ogr.CreateGeometryFromWkt("POINT (2618579 1244235)")
print(f"Ausgangspunkt LV95: {point.ExportToWkt()}")
point.Transform(transformationdef)
point2.Transform(transformationdef)
point2.Transform(retransformationdef)

pointWGS84 = ogr.CreateGeometryFromWkt(point.ExportToWkt())
print(f"Transformierter Punkt in WGS84: {pointWGS84.ExportToWkt()}")
print(f"Rücktransformierter Punkt in LV95: {point2.ExportToWkt()}")

x=str(point.ExportToWkt()).split('(')[1].split(' ')[0]
y=str(point.ExportToWkt()).split('(')[1].split(' ')[1].split(',')[0]
print(x,y)
```

Aufgabe:

Erstelle je eine `*.csv` Datei, welche die Zentroide aller Gemeinden des Kantons Solothurn enthält. Die eine Datei soll die Daten im nativen räumlichen Referenzsystem enthalten und die andere im Referenzsystem WGS84.

```
In [ ]: from osgeo import osr

wpath = '../Data/'
source = osr.SpatialReference()
source.ImportFromEPSG(21781)
destination = osr.SpatialReference()
destination.ImportFromEPSG(4326)
transformationdef = osr.CoordinateTransformation(source, destination)

# Extraktion aller Zentroide der Gemeinden
centroidFlativ = os.path.join(wpath, 'centroidsGemSoNativ.csv')
centroidFlWGS84 = os.path.join(wpath, 'centroidsGemSoWGS84.csv')
with open(centroidFlativ, 'w', encoding='utf-8') as centroidFlativ:
    with open(centroidFlWGS84, 'w', encoding='utf-8') as centroidFlWGS84:
        centroidFlativ.write("GemName,X,Y\n")
        centroidFlWGS84.write("GemName,lon,lat\n")
        for feature in layer:
            geometry = feature.GetGeometryRef()
            minEasting,maxEasting,minNorthing,maxNorthing = geometry.GetEnvelope()
            centerX = (minEasting + maxEasting)/2
            centerY = (minNorthing + maxNorthing)/2
```

```

gemName = feature.GetField('gmde_name')
ftrInfoNativ = f"{gemName},{centerX},{centerY}\n"
centroidFlNativ.write(ftrInfoNativ)

# Umprojektion
pointWGS84 = ogr.CreateGeometryFromWkt(f"POINT ({centerX} {centerY})")
pointWGS84.Transform(transformationdef)
ftrInfoWGS84 = f"{gemName},{pointWGS84.GetY()},{pointWGS84.GetX()}\n"
centroidFlWGS84.write(ftrInfoWGS84)

print(f"Die Dateien {centroidFlNativ} und {centroidFlWGS84} wurden erstellt.")

```

In []: *# Alternative Lösung*

```

import csv
from osgeo import ogr

wpath = '../Data/'
source = ogr.SpatialReference()
source.ImportFromEPSG(21781)
destination = ogr.SpatialReference()
destination.ImportFromEPSG(4326)
transformationdef = ogr.CoordinateTransformation(source, destination)

dictNativ = [['Gemeinde', 'Centroid']]
dictWGS84 = [['Gemeinde', 'Centroid']]
for feature in layer:
    name = feature.GetField("name")
    polygon = feature.GetGeometryRef()
    centroidNativ = polygon.Centroid()
    dictNativ.append([name, centroidNativ])

# Umprojektion
centroidWGS84 = ogr.CreateGeometryFromWkt(f"POINT ({centroidNativ.GetX()} {centroidNativ.GetY()})")
centroidWGS84.Transform(transformationdef)
dictWGS84.append([name, centroidWGS84])

with open(os.path.join(wpath, 'GemSOCenterNativ.csv'), 'w', newline='', encoding='utf-8') as csvfile:
    csvwriter = csv.writer(csvfile)
    csvwriter.writerows(dictNativ)

with open(os.path.join(wpath, 'GemSOCenterWGS84.csv'), 'w', newline='', encoding='utf-8') as csvfile:
    csvwriter = csv.writer(csvfile)
    csvwriter.writerows(dictWGS84)

```

Rasterdaten

Nachdem Vektordaten analysiert wurden, werden im Folgenden Rasterdaten ausgewertet.

Dazu dient das Modul `gdal` von `osgeo`. Ein Datensatz, also ein Bild, kann mit der Methode `open` geöffnet und anschliessend analysiert werden. Dazu dienen beispielsweise die Eigenschaften `RasterXSize` (Anzahl Spalten) und `RasterYSize` (Anzahl Zeilen) und `RasterCount` (Anzahl Bänder).

Ausführliche Informationen sind zu finden unter https://gdal.org/en/stable/tutorials/raster_api_tut.html.

Es soll nun als erstes die Rasterdatei `ortho14_5m_rgb_solothurn.tif` geöffnet und hinsichtlich der erwähnten Grössen untersucht werden.

```
wpath = '../Data/'
rasFl = os.path.join(wpath, 'ortho14_5m_rgb_solothurm.tif')

ds = gdal.Open(rasFl)
cols = ds.RasterXSize
rows = ds.RasterYSize
bands = ds.RasterCount

print(f"Anzahl Spalten: {cols}, Anzahl Zeilen: {rows}, Anzahl Bänder: {bands}")
```

Aufgabe:

Analysiere die Datei `ortho14_5m_rgb_solothurm.tif` hinsichtlich deren Dimensionen und bezüglich der Koordinaten des Ursprungs und ermittle für jedes Band die minimalen und maximalen radiometrischen Werte.

```
In [ ]: from osgeo import gdal
rb = os.path.join(wpath, 'ortho14_5m_rgb_solothurm.tif')
wpath = '../Data/'
def getDifferentRasterInformation(rasFl):
    ds = gdal.Open(rasFl)
    cols = ds.RasterXSize
    rows = ds.RasterYSize
    bands = ds.RasterCount

    print(f"Anzahl Spalten: {cols}, Anzahl Zeilen: {rows}, Anzahl Bänder: {bands}")

    addInfo = ds.GetGeoTransform()
    orig = f"Ursprung: x = {addInfo[0]}, {addInfo[3]}"
    psize = f"Pixelgröße: x = {addInfo[1]}, y = {addInfo[5]}"
    rot = f"Rotation: Achse 1 = {addInfo[2]}, y = {addInfo[4]}"
    print(f"{orig}\n{psize}\n{rot}\n")

    #Bandinformationen
    for bandnr in range(bands):
        band = ds.GetRasterBand(bandnr+1)
        print ('Band-Typ: ', gdal.GetDataTypeName(band.DataType))

        if not band is None:
            min = band.GetMinimum()
            max = band.GetMaximum()
            ct = band.GetColorTable()
            if not ct is None:
                print ('Band hat ', ct, ' Farbpalette.')

        if min is None or max is None:
            (min,max) = band.ComputeRasterMinMax(1)

        print ('Min=%.3f, Max=%.3f' % (min,max))

        if band.GetOverviewCount() > 0:
            print ('Band hat ', band.GetOverviewCount(), ' Übersichten.')

        if not band.GetRasterColorTable() is None:
            print ('Band hat eine Farbtabelle mit ', \
                    band.GetRasterColorTable().GetCount(), ' Einträgen.')
```

Die Analyse funktioniert auch für ein nicht räumliches Bild:

No description has been provided for this image

```
In [ ]: scndPic = os.path.join(wpath, 'donaldrump.jpeg')
        getDifferentRasterInformation(scndPic)
```

Radiometrischer Wert anhand Koordinate auslesen

Es kann durchaus vorkommen, dass ein gegebener Vektordatensatz von Punkten gegeben ist und für diese der Radiometrische Wert eines darunter liegenden Rasters ausgelesen werden soll. Ein erstes einfaches Beispiel wie der Wert eines Punktes ermittelt wird, findet sich in folgendem Code-Beispiel:

```
import os, sys, numpy, time, csv
from osgeo import gdal
from osgeo.gdalconst import *

#register all of the drivers
gdal.AllRegister()

#open the image
ds = gdal.Open('Data/ortho14_5m_rgb_solothurm.tif', GA_ReadOnly)

#get image size
rows = ds.RasterYSize
cols = ds.RasterXSize
bands = ds.RasterCount

#get georeference info
transform = ds.GetGeoTransform()
x0origin = transform[0]
y0origin = transform[3]
pixelWidth = transform[1]
pixelHeight = transform[5]

x = 594000.0
y = 229500.0

#compute pixel offset
xOffset = int((x - x0origin) / pixelWidth)
yOffset = int((y - y0origin) / pixelHeight)

band = ds.GetRasterBand(1)

#read data and add the value to the string
data = band.ReadAsArray(xOffset, yOffset, 1, 1)
value = data[0,0]
print(data)
```

```
In [ ]: import os, sys, numpy, time, csv
        from osgeo import gdal
        from osgeo.gdalconst import *

        wpath = '../Data/'
        # register all of the drivers
        gdal.AllRegister()
        # open the image
        ds = gdal.Open(wpath, 'ortho14_5m_rgb_solothurm.tif'), GA_ReadOnly)
```

```

# get image size
rows = ds.RasterYSize
cols = ds.RasterXSize
bands = ds.RasterCount
# get georeference info
transform = ds.GetGeoTransform()
xOrigin = transform[0]
yOrigin = transform[3]
pixelWidth = transform[1]
pixelHeight = transform[5]

x = 594000.0
y = 229500.0
# compute pixel offset
xOffset = int((x - xOrigin) / pixelWidth)
yOffset = int((y - yOrigin) / pixelHeight)

band = ds.GetRasterBand(1)
#read data and add the value to the string
data = band.ReadAsArray(xOffset, yOffset, 1, 1)
value = data[0,0]
print(data)

```

Aufgabe:

Für alle Zentroide der Gemeinden des Kantons Solothurn sollen die radiometrischen Werte aller Bänder des Satellitenbildes 'ortho14_5m_rgb_solothurn.tif' ermittelt und in eine CSV Datei geschrieben werden. Die CSV Datei soll folgende Spalten haben, die zu befüllen sind beim Export:

```
'X , Y , xOffset , yOffset , Wert Band 1 , Wert Band 2 , Wert Band 3'
```

```

In [ ]: # obtained from http://www.gis.usu.edu/~chrisg/python/2009/lectures/ospyslides4.pdf
# script to get pixel values at a set of coordinate by reading in one pixel at a time

import os, sys, numpy, time, csv
from osgeo import gdal
from osgeo.gdalconst import *

wpath = '../Data/'
# start timing
startTime = time.time()
# coordinates to get pixel values for
#xValues = [594000.0, 604000.0, 613500.0, 599594.0]
#yValues = [229500.0, 231000.0, 222800.0, 226081.0]

centrPnts = os.path.join(wpath, "centroidsGemSoNativ.csv")
with open(centrPnts, 'r', encoding='utf-8') as centroidFl:
    cpwri = os.path.join(wpath, 'centroidsGemSoWithRasterInformation.csv')
    with open(cpwri, 'w', encoding='utf-8') as cpwriFl:

        # register all of the drivers
        gdal.AllRegister()
        # open the image
        ds = gdal.Open(os.path.join(wpath, 'ortho14_5m_rgb_solothurn.tif'), GA_ReadOnly)
        if ds is None:
            print('Could not open image')
            sys.exit()

```

```

# get image size
rows = ds.RasterYSize
cols = ds.RasterXSize
bands = ds.RasterCount
# get georeference info
transform = ds.GetGeoTransform()
xOrigin = transform[0]
yOrigin = transform[3]
pixelWidth = transform[1]
pixelHeight = transform[5]
outStr = 'X , Y , xOffset , yOffset , Wert Band 1 , Wert Band 2 , Wert Band 3'
#print(outStr)
cpwriFl.write(outStr)

output=False
cnt = 0
lineCnt = 0

# loop through the coordinates
for lines in centroidFl:
    # get x,y
    if lineCnt > 0:
        x = float(lines.split(",")[1])
        y = float(lines.split(",")[2])
        # compute pixel offset
        xOffset = int((x - xOrigin) / pixelWidth)
        yOffset = int((y - yOrigin) / pixelHeight)

        # create a string to print out
        s = str(x) + ', ' + str(y) + ', ' + str(xOffset) + ', ' + str(yOffset)
        # loop through the bands
        for j in range(bands):
            band = ds.GetRasterBand(j+1) # 1-based index
            #read data and add the value to the string
            data = band.ReadAsArray(xOffset, yOffset, 1, 1)
            try:
                value = data[0,0]
                #print(data)
                #value2 = numpy.median(data)
                s = s + str(value) + ', '
                output=True
            except:
                output=False
            pass

        #print out the data string
        if output:
            print(s)
            cpwriFl.write(f"{s[:-1]}\n")
            cnt += 1
        lineCnt += 1
# figure out how long the script took to run
endTime = time.time()
print()
print('The script took %.3f seconds' %(endTime - startTime))
print(f"{cnt} Punkte innerhalb des Rasterbildes von {lineCnt}")

```

Höhenmodellinformationen ableiten

Über die Kommandozeile können mit dem Befehl `gdal_contour` wichtige abgeleitete Produkte eines Höhenmodells erstellt werden. Zu diesen zählen:

- Konturlinien/Höhenlinien

- Exposition
- Neigung
- Schummerung

Ein möglicher Aufruf für Höhenlinien könnte wie folgt sein:

```
gdal_contour -a hoehe -i 150 Data/Elevation_raster.tif Data/contour150.shp
```

Nähere Informationen dazu sind unter https://gdal.org/programs/gdal_contour.html zu finden.

Es sollen nun die erwähnten Produkte anhand des Höhenmodells `Elevation_raster.tif` erstellt werden.

```
In [ ]: cmd = 'gdal_contour -a hoehe -i 150 ../Data/Elevation_raster.tif ../Data/contour150.shp'
os.system(cmd)
```

```
In [ ]: rcmd = ["gdal_contour", "-a", "hoehe", "-i", "150", "../Data/Elevation_raster.tif", "-o", "contour150.shp"]
runsubprocess(rcmd)
```

```
In [ ]: cmdList = []
wpath = "../Data/"
cmdList.append(f'gdaldem slope {wpath}Elevation_raster.tif {wpath}Ele_slope.tif -p')
cmdList.append(f'gdaldem aspect {wpath}Elevation_raster.tif {wpath}Ele_aspect.tif')
cmdList.append(f'gdaldem hillshade {wpath}Elevation_raster.tif {wpath}Ele_hillshade.tif')
for cmd in cmdList:
    os.system(cmd)
    print(cmd)
```

```
In [ ]: cmdList = []
wpath = "../Data/"
cmdList.append(["gdaldem", "slope", wpath+"Elevation_raster.tif", wpath+"Ele_slope.tif"])
cmdList.append(["gdaldem", "aspect", wpath+"Elevation_raster.tif", wpath+"Ele_aspect.tif"])
cmdList.append(["gdaldem", "hillshade", wpath+"Elevation_raster.tif", wpath+"Ele_hillshade.tif"])
for cmd in cmdList:
    runsubprocess(cmd)
```

Aufgabe:

Erstelle ein Pythonskript, das basierend auf einem gegebenen Höhenmodellraster die erwähnten Produkte erstellt.

```
In [ ]: import os, sys
from osgeo import gdal
from osgeo.gdalconst import *

wpath = "../Data/"
# register all of the drivers
gdal.AllRegister()

#Open Rasterfile
fn = os.path.join(wpath, 'worldmap.jpg')
ds = gdal.Open(fn)
if ds is None:
    print ('Datensatz %s konnte nicht geöffnet werden' %fn)
    sys.exit(1)
```

```

#os.system('gdalinfo %s' %fn)
rcmd = ["gdalinfo", fn]
runsubprocess(rcmd)

#translatecommand = f'gdal_translate -projwin 1680 170 2200 550 %s {wpath}europe.tif'
#print ("command to run: %s" %translatecommand )
#os.system(translatecommand)
rcmd = ["gdal_translate", "-projwin", "1680", "170", "2200", "550", fn, wpath+"europe.tif"]
runsubprocess(rcmd)

#kleinere Kopie von Europa
#translatecommand = 'gdal_translate -projwin 1680 170 2200 550 -outsize 50%% 50%% %s'
#print ("command to run: %s" %translatecommand)
#os.system(translatecommand)
rcmd = ["gdal_translate", "-projwin", "1680", "170", "2200", "550", "-outsize", "50%", "50%", fn, wpath+"europe.tif"]
runsubprocess(rcmd)

```

[Top](#)

Teil III Spezifische Aufgaben

Geodaten schreiben

Geodaten können mit `OGR/GDAL` nicht nur analysiert, sondern auch erstellt werden. Im Folgenden werden zuerst Vektordaten, danach auch Rasterdaten erstellt.

Vektordaten erstellen

Das Vorgehen um Vektordaten zu erstellen ähnelt dem Vorgehen bei der Analyse von Vektordaten. Dabei sind gewisse Mindestanforderungen zu definieren, damit ein neuer Vektordatensatz erstellt werden kann. Zu diesen Mindestanforderungen gehören:

- Name der Datei und des Layers
- Speicherort der Datei
- Format des Datensatzes
- Räumliches Referenzsystem des Datensatzes
- Attributdefinitionen (Namen, Datentyp, Wertebereich) des Datensatzes
- Für die Objekte:
 - Attributwerte
 - Geometrien

Um diese Eigenschaften zu definieren werden die Module `OGR` und `OSR` benötigt.

Die Definition des Geodatenformats erfolgt über die Definition des Treibers mittels `ogr.GetDriverByName()`. Dabei werden die Methoden `CreateDataSource()` und deren Methode `CreateLayer()` verwendet. Der Name und Ablageort der Daten erfolgt über Standard-Pythonbefehle. Die Definition der Attribute geschieht mittels Felddefinitionen `ogr.FieldDefn()`. Ein konkretes Beispiel dazu für die Definition eines Text-Attributs lautet:

```

fieldDef = ogr.FieldDefn('name',ogr.OFTString)
fieldDef.SetWidth(50)
destinationLyr.CreateField(fieldDef)

```

Die Geometriedefinition ist stark hierarchisch aufgebaut. So werden zuerst die Stützpunkte definiert, danach die inneren Geometrien wie beispielsweise ein linearer Ring und schliesslich die Endgeometrie in Form eines Polygons. Als Beispielcode diene hier:

Featuredefinition – Erstellung eines Eintrags in die erstellte

Layerstruktur

```
minEasting = 7.5
maxEasting = 7.6
minNorthing = 46.5
maxNorthing = 46.6
```

Linearer Ring erstellen

```
lR = ogr.Geometry(ogr.wkbLinearRing)
```

Stützpunkte definieren

```
lR.AddPoint(minEasting, minNorthing)
lR.AddPoint(maxEasting, minNorthing)
lR.AddPoint(maxEasting, maxNorthing)
lR.AddPoint(minEasting, maxNorthing)
lR.AddPoint(minEasting, minNorthing)
```

Instanziierung der Geometrie als WkbPolygon

```
sqr = ogr.Geometry(ogr.wkbPolygon)
```

Zuweisen der Geometrie

```
sqr.AddGeometry(lR)
```

Zum Schluss wird das Objekt selbst mit den zuvor definierten Werten instanziiert. Dies geschieht beispielsweise wie folgt:

```
ftrName = "Wert eines Textes"
```

Feature mit Attribut- und Geometriedefinition

```
sqrFtr = ogr.Feature(destinationLyr.GetLayerDefn())
sqrFtr.SetGeometry(sqr)
sqrFtr.SetField("name", ftrName)
sqrFtr.SetField("bemerkung", "Hurra, ich bin ein Quadrat!")
sqrFtr.SetField("wert", 9)
```

Feature dem Layer hinzufügen

```
destinationLyr.CreateFeature(sqrFtr)
```

Last but not least muss das Feature wieder freigegeben werden und dazu dient der Befehl

`Destroy()`, der nach Abschluss der Geometrieerstellung sowohl auf das Objekt/Feature als auch den Layer anzuwenden ist.

```
In [ ]: from osgeo import ogr
        from osgeo import osr

        # Räumliche Referenzsystem setzen
        srs = osr.SpatialReference()
        #srs.SetWellKnownGeogCS('WGS84')
        #srs.SetFromUserInput("EPSG:4326")
        srs.ImportFromEPSG(4326)

        # Alternative: srs aus existierendem Layer verwenden
        #srs.ImportFromProj4(layer.GetSpatialRef().ExportToProj4())

        # Datei erstellen vom Typ Esri Shapefile
        driver = ogr.GetDriverByName("ESRI Shapefile")
        destFlNm = os.path.join(wpath, "myFirstLyr.shp")

        ...

        #GeoJSON
        driver = ogr.GetDriverByName("GeoJSON")
        destFlNm = os.path.join("Data", "myFirstLyr.geojson")

        #GeoPackage
        driver = ogr.GetDriverByName("GPKG")
        destFlNm = os.path.join("Data", "myFirstLyr.gpkg")
```

```

'''
if os.path.exists(destFlNm):
    driver.DeleteDataSource(destFlNm)
destinationFile = driver.CreateDataSource(destFlNm)
destinationLyr = destinationFile.CreateLayer("lyr", srs)

# GeoPackage Test
#destinationFile2 = driver.Open(destFlNm)
destinationLyr2 = destinationFile.CreateLayer("lyr2", srs)

# Attributdefinition: 3 Attribute: name (String), bemerkung (String), wert (Integer)
fieldDef = ogr.FieldDefn('name', ogr.OFTString)
fieldDef.SetWidth(50)
destinationLyr.CreateField(fieldDef)
fieldDef = ogr.FieldDefn('bemerkung', ogr.OFTString)
fieldDef.SetWidth(150)
destinationLyr.CreateField(fieldDef)
fieldDef = ogr.FieldDefn('wert', ogr.OFTInteger)
destinationLyr.CreateField(fieldDef)
destinationLyr2.CreateField(fieldDef)

fieldDef = ogr.FieldDefn('flaeche', ogr.OFTReal)
destinationLyr.CreateField(fieldDef)

# Featuredefinition – Erstellung eines Eintrags in die erstellte Layerstruktur
ftrName = 'square'
minEasting = 7.5
maxEasting = 7.6
minNorthing = 46.5
maxNorthing = 46.6

# Linearer Ring erstellen
lR = ogr.Geometry(ogr.wkbLinearRing)

# Stützpunkte definieren
lR.AddPoint(minEasting, minNorthing)
lR.AddPoint(maxEasting, minNorthing)
lR.AddPoint(maxEasting, maxNorthing)
lR.AddPoint(minEasting, maxNorthing)
lR.AddPoint(minEasting, minNorthing)

# Instanziierung der Geometrie als WkbPolygon
sqr = ogr.Geometry(ogr.wkbPolygon)

# Zuweisen der Geometrie
sqr.AddGeometry(lR)
ftrarea = abs(sqr.GetArea())

# Feature mit Attribut- und Geometriedefinition
sqrFtr = ogr.Feature(destinationLyr.GetLayerDefn())
sqrFtr.SetGeometry(sqr)
sqrFtr.SetField("name", ftrName)
sqrFtr.SetField("bemerkung", "Hurra, ich bin ein quadrat!")
sqrFtr.SetField("wert", 9)
sqrFtr.SetField("flaeche", ftrarea)
# Feature dem Layer hinzufügen
destinationLyr.CreateFeature(sqrFtr)
#destinationLyr2.CreateFeature(sqrFtr)

# Freigabe des Featureobjekts und der Datei
sqrFtr.Destroy()
destinationFile.Destroy()

```

Aufgabe:

Erstelle einen Polygonlayer, der mindestens ein Objekt mit einer Insel hat.

```
In [ ]: from osgeo import ogr, osr
import os

#Definition SRS
srs = osr.SpatialReference()
srs.SetWellKnownGeogCS('WGS84')

wPathFl = os.path.join(wpath, 'Insellayer.shp')
#Erstellen der neuen Ebene/Layer
driver = ogr.GetDriverByName("Esri Shapefile")
#driver = ogr.GetDriverByName("GeoJSON")
if os.path.exists(wPathFl):
    driver.DeleteDataSource(wPathFl)
destinationFile = driver.CreateDataSource(wPathFl)
#destinationFile = driver.CreateDataSource(wPathFl)
destinationLayer = destinationFile.CreateLayer("Layer", srs)

#Festlegung der Attribute
fieldDef = ogr.FieldDefn('Id', ogr.OFTInteger)
destinationLayer.CreateField(fieldDef)
fieldDef = ogr.FieldDefn("Name", ogr.OFTString)
fieldDef.SetWidth(80)
destinationLayer.CreateField(fieldDef)
fieldDef = ogr.FieldDefn("Bemerkung", ogr.OFTString)
fieldDef.SetWidth(100)
destinationLayer.CreateField(fieldDef)

#Erstellen eines Features
ftrName = "Erstes Feature"
ftrBem = "Dies ist mein erstes selbst erstelltes Features"

#geometry = feature.GetGeometryRef()
minEasting = 10
maxEasting = 20
minNorthing = 15
maxNorthing = 25

#Definition des OGR Geometrieobjekts als LinearRing
linearRing = ogr.Geometry(ogr.wkbLinearRing)
#Hinzufügen der Stützpunkte des LinearRing
linearRing.AddPoint(minEasting, minNorthing)
linearRing.AddPoint(maxEasting, minNorthing)
linearRing.AddPoint(maxEasting, maxNorthing)
linearRing.AddPoint(minEasting, maxNorthing)
linearRing.AddPoint(minEasting, minNorthing)

#Definition des OGR Geometrieobjekts als LinearRing
linearRing2 = ogr.Geometry(ogr.wkbLinearRing)
#Hinzufügen der Stützpunkte des LinearRing
linearRing2.AddPoint(minEasting+3, minNorthing+3)
linearRing2.AddPoint(maxEasting-3, minNorthing+3)
linearRing2.AddPoint(maxEasting-3, maxNorthing-3)
linearRing2.AddPoint(minEasting+3, maxNorthing-3)
linearRing2.AddPoint(minEasting+3, minNorthing+3)
```

```

#Definition des OGR Geometrieobjekts als LinearRing
linearRing3 = ogr.Geometry(ogr.wkbLinearRing)
#Hinzufügen der Stützpunkte des LinearRing
linearRing3.AddPoint(minEasting+30, minNorthing+30)
linearRing3.AddPoint(maxEasting+30, minNorthing+30)
linearRing3.AddPoint(maxEasting+30, maxNorthing+30)
linearRing3.AddPoint(minEasting+30, maxNorthing+30)
linearRing3.AddPoint(minEasting+30, minNorthing+30)

#Definition des OGR Geometrieobjekts als LinearRing
linearRing4 = ogr.Geometry(ogr.wkbLinearRing)
#Hinzufügen der Stützpunkte des LinearRing
linearRing4.AddPoint(minEasting+8, minNorthing+8)
linearRing4.AddPoint(maxEasting+25, minNorthing+8)
linearRing4.AddPoint(maxEasting+25, maxNorthing+25)
linearRing4.AddPoint(minEasting+8, maxNorthing+25)
linearRing4.AddPoint(minEasting+8, minNorthing+8)

#Instanzieren der Geometrie als WKBPolygon ins sqr Objekt
sqr = ogr.Geometry(ogr.wkbPolygon)
#Zuweisen der Geometrie zum instanzierten Objekt
sqr.AddGeometry(linearRing)
sqr.AddGeometry(linearRing2)
sqr.AddGeometry(linearRing3)
sqr.AddGeometry(linearRing4)
#Neues Feature erhält Attributdefinition
sqrfeature = ogr.Feature(destinationLayer.GetLayerDefn())
#Neues Feature erhält Geometrie
sqrfeature.SetGeometry(sqr)
#Neues Feature erhält für das Attribut Name den Wert "Erstes Feature"
sqrfeature.SetField("Id", 1)
sqrfeature.SetField("Name", ftrName)
sqrfeature.SetField("Bemerkung", ftrBem)
#Erstellung des Features im neuen Layer
destinationLayer.CreateFeature(sqrfeature)
sqrfeature.Destroy()

print(f"Erstellung abgeschlossen. Die Datei {wPathFl} wurde erstellt.")
destinationFile.Destroy()

```

Nebst der Erstellung einer neuen Vektordaten-Datei können auch neue Objekte einem bestehenden Datensatz hinzugefügt werden.

Aufgabe:

Füge der zuvor erstellten Vektordatei ein neues Objekt hinzu mit zufälligen Koordinaten der Stützpunkte.

In []: # Geometrie zu existierendem Datensatz hinzufügen

```

from osgeo import ogr
from osgeo import osr
import random
from shapely import Polygon

path = os.path.join(wpath, "myFirstLyr.shp")
driver = ogr.GetDriverByName("ESRI Shapefile")
ds = driver.Open(path, 1)

```

```

definition = layer.GetLayerDefn()

print("Name | Type")
for i in range(definition.GetFieldCount()):
    name = definition.GetFieldDefn(i).GetName()
    type = definition.GetFieldDefn(i).GetTypeName()
    print(name, ' | ', type)

feature = ogr.Feature(definition)

feature.SetField("Name", "Ergänzttes Objekt")
feature.SetField("Bemerkung", "Objekt zu bestehendem Layer hinzugefügt")
feature.SetField("Wert", "99")

# Erzeuge zufällige Geometrie
# Anzahl der Stützpunkte zufällig zwischen 8 und 15 wählen
num_points = random.randint(8, 15)

# Generiere zufällige Stützpunkte (Longitude: 7.5–7.6, Latitude: 47.5–47.7)
coordinates = [
    (
        random.uniform(7.5, 7.6), # Zufällige Longitude
        random.uniform(47.5, 47.7) # Zufällige Latitude
    )
    for _ in range(num_points)
]

# Schließe das Polygon, indem der erste Punkt erneut hinzugefügt wird
coordinates.append(coordinates[0])

# Erstelle das Polygon mit Shapely
polygon = Polygon(coordinates)

# Erzeuge das Well-known-Text (WKT)
wkt = polygon.wkt

#wkt = f"POLYGON(({s}))"
print(wkt)
polygon = ogr.CreateGeometryFromWkt(wkt)
feature.SetGeometry(polygon)
layer.CreateFeature(feature)
feature.Destroy()
ds.Destroy()

```

Rasterdaten schreiben

Analog zu Vektordaten sollen nun auch Rasterdaten erzeugt, dh neu erstellt werden.

Auch hier sind wieder diverse Mindestinformationen zu definieren:

- Dateiname und Ablageort
- Format der Rasterdatei
- Dimension der Rasterdatei
- Anzahl Bänder der Rasterdatei
- Räumliches Bezugssystem der Rasterdatei

Das Format der Rasterdatei wird bestimmt durch den Befehl `gdal.GetDriverByName()`.

Die Dimension der Rasterdatei wird bestimmt durch den Befehl `Create()`, der eine Methode des zuvor instanziierten Treiberobjektes (Format) ist. Die Parameter, die dabei mitgegeben werden müssen sind der Ablageort und Name der Datei, deren Grösse in Spalten und Zeilen (als Anzahl

Pixel) und die Anzahl der Bänder.

Das Räumliche Bezugssystem der Rasterdatei wird bestimmt durch den Befehl

`SpatialReference()` des Moduls `osr`. Wird ein Objekt so instanziiert, kann mit der Methode `ImportFromEPSG()` das entsprechende Referenzsystem zugewiesen werden (bspw. 4326 für WGS84).

Schliesslich müssen noch die eigentlichen Nutzdaten, dh. Pixelwerte den einzelnen Bändern des Bildes zugewiesen werden. Dazu müssen entsprechende Matrizen, welche der Dimension des Bildes entsprechen, übernommen oder gebildet werden, die dann dem zugehörigen Kanal zugewiesen werden. Mit dem Modul `numpy` können zu Testzwecken Daten simuliert werden. Dies kann beispielsweise für eine 10x10 Matrix wie folgt aussehen:

```
matrix1 = np.random.randint(0, 256, (10, 10))
```

Schliesslich müssen die erstellten Werte den Bändern und diese dann dem Bild zugewiesen werden (Beispiel für das Band 1): `ds_create.GetRasterBand(1).WriteArray(matrix1)`

Nach der Erstellung der Rasterdatei, ist diese sauber zu schliessen bzw. wieder freizugeben:

```
ds_create = None
```

```
In [ ]: from osgeo import gdal
        from osgeo import osr
        import numpy as np
        import matplotlib.image as mpimg
        import matplotlib.pyplot as plt
        import matplotlib as mpl

        cmap = mpl.colormaps['viridis']

        fn_create = os.path.join(wpath, "gdalCreateRaster3.tif") # filename for new raster
        driver_gtiff = gdal.GetDriverByName('GTiff')
        ds_create = driver_gtiff.Create(fn_create, xsize=10, ysize=10, bands=3, eType=gdal.GDT_Byte)

        srs = osr.SpatialReference()
        srs.ImportFromEPSG(21781)
        ds_create.SetProjection(srs.ExportToWkt())

        geot_create = [600000, 10.0, 0.0, 200000, 0.0, -10.0]
        ds_create.SetGeoTransform(geot_create)
        print(ds_create.GetGeoTransform())

        data_createR = np.random.randint(0, 256, (10, 10))
        data_createG = np.random.randint(0, 256, (10, 10))
        data_createB = np.random.randint(0, 256, (10, 10))

        ds_create.GetRasterBand(1).WriteArray(data_createR) # write the array to the raster
        ds_create.GetRasterBand(1).SetNoDataValue(0) # set the no data value
        ds_create.GetRasterBand(2).WriteArray(data_createG) # write the array to the raster
        ds_create.GetRasterBand(2).SetNoDataValue(0) # set the no data value
        ds_create.GetRasterBand(3).WriteArray(data_createB) # write the array to the raster
        ds_create.GetRasterBand(3).SetNoDataValue(0) # set the no data value

        ds_create = None # properly close the raster
```

Um das Bild anzuzeigen, kann mit dem Modul `matplotlib.image` die eben erstellte Rasterdatei geöffnet und ausgegeben werden (Befehl via `matplotlib.pyplot`).

```
In [ ]: import matplotlib.image as mpimg
        import matplotlib.pyplot as plt
```

```
import matplotlib as mpl
```

```
image = mpimg.imread(fn_create)
# plot the data values we created
plt.figure(figsize=(10, 10))
plt.imshow(image)
plt.colorbar()
```

Aufgabe:

Erstelle einen Rasterdatei der Dimension 50x30, welche in den ersten 10 Zeilen nur rote, in den zweiten 10 Zeilen nur grüne und in den letzten 10 Zeilen blaue Pixel aufweist.

```
In [ ]: from osgeo import gdal
from osgeo import osr
import numpy as np
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import matplotlib as mpl

cmap = mpl.colormaps['viridis']

fn_create = os.path.join(wpath, "gdalCreateRaster3.tif") # filename for new raster

driver_gtiff = gdal.GetDriverByName('GTiff')
ds_create = driver_gtiff.Create(fn_create, xsize=50, ysize=30, bands=3, eType=gdal.GDT_Byte)

srs = osr.SpatialReference()
srs.ImportFromEPSG(21781)
ds_create.SetProjection(srs.ExportToWkt())

geot_create = [600000, 30.0, 0.0, 200000, 0.0, -50.0]
ds_create.SetGeoTransform(geot_create)
print(ds_create.GetGeoTransform())

data_createR = np.zeros((30, 50))
data_createR[:10, :50] = 255 # values to 1, leave outer as 0 (no data)
data_createG = np.zeros((30, 50))
data_createG[10:20, :50] = 255 # values to 1, leave outer as 0 (no data)
data_createB = np.zeros((30, 50))
data_createB[20:30, :50] = 255 # values to 1, leave outer as 0 (no data)

ds_create.GetRasterBand(1).WriteArray(data_createR) # write the array to the raster
ds_create.GetRasterBand(1).SetNoDataValue(0) # set the no data value
ds_create.GetRasterBand(2).WriteArray(data_createG) # write the array to the raster
ds_create.GetRasterBand(2).SetNoDataValue(0) # set the no data value
ds_create.GetRasterBand(3).WriteArray(data_createB) # write the array to the raster
ds_create.GetRasterBand(3).SetNoDataValue(0) # set the no data value

ds_create = None # properly close the raster

image = mpimg.imread(fn_create)
# plot the data values we created
plt.figure(figsize=(30, 50))
plt.imshow(image)
plt.colorbar()
```


Aufgabe:

Erstelle ein Skript, das die Eingabe eines Gemeindennamens einer Gemeinde im Kanton Solothurn erlaubt und anschliessend dessen Stützpunkte als Punktdatensatz im Format .csv ausgibt.

Lösung für das Beispiel der Gemeinde Oensingen:



No description has been provided for this image

```
In [ ]: import osgeo.ogr as ogr
import sys

def extractPoints(geometry, expFl):
    for (i) in range(geometry.GetPointCount()):
        x,y,z = geometry.GetPoint(i)
        expFl.write( f"{i+1},{x},{y}\n")

    for i in range(geometry.GetGeometryCount()):
        extractPoints(geometry.GetGeometryRef(i),expFl)

gemname = input("Gemeindename:")

logFl = f"{wpath}/{gemname}.csv"
exportfile = open(logFl, "w")

shapefile = ogr.Open(os.path.join(wpath,"Gemeinden_Solothurn.shp"))
if shapefile is None:
    exportfile.write( "Datensatz konnte nicht geoeffnet werden.\n" + "\n")
    sys.exit( 1 )

layer = shapefile.GetLayer(0)
#geometry = feature.GetGeometryRef()

#Gemeindegeometry extrahieren:
geometry = None
for feature in layer:
    if feature.GetField("NAME") == gemname:
        geometry = feature.GetGeometryRef()
        break

if geometry is None:
    exportfile.write( "*" * 20 + "\n")
    exportfile.write( "Fuer %s konnte keine Geometrie ermittelt werden." %gemname +
    exportfile.write( "*" * 20 + "\n")
    sys.exit( 1 )

exportfile.write( "pid,x,y\n")
extractPoints(geometry,exportfile)
exportfile.write( "-" * 50 + "\n")
print(f"Ausgabe siehe {logFl}")
exportfile.close()
```

[Top](#)

Teil IV Spezifische Aufgaben

Aufgabe:

Erstelle ein Skript, das einen neuen Vektordatensatz erstellt, der die MBRs (minimum bounding rectangles) aller Gemeinden des Kantons Solothurn erstellt.

Lösung:



No description has been provided for this image

```
In [ ]: import osgeo.ogr as ogr
import osgeo.osr as osr
import osgeo.gdal
import osgeo.gdalconst
import sys

sourcelayer = os.path.join(wpath, "Gemeinden_Solothurn.shp")
destinationlayername = os.path.join(wpath, "gemSoMBR.shp")
destinationformat = "ESRI Shapefile"
sourcefieldname = "name"

shapefile = osgeo.ogr.Open(sourcelayer)
sourcelayer = shapefile.GetLayer(0)
srs = osr.SpatialReference()
srs.ImportFromProj4(sourcelayer.GetSpatialRef().ExportToProj4())

driver = osgeo.ogr.GetDriverByName(destinationformat)
destinationFile = driver.CreateDataSource(destinationlayername)
destinationLayer = destinationFile.CreateLayer(destinationlayername[0:len(destinationlayername)], srs, osgeo.ogr.OFWrite)

#Create Field to store the name
fieldDef = osgeo.ogr.FieldDefn(sourcefieldname, osgeo.ogr.OFTString)
fieldDef.SetWidth(100)
destinationLayer.CreateField(fieldDef)

feature = sourcelayer.GetNextFeature()
while feature:
    #Get value of Feature-Name
    ftrName = feature.GetField(sourcefieldname)
    #Get MBR
    geometry = feature.GetGeometryRef()
    minEasting, maxEasting, minNorthing, maxNorthing = geometry.GetEnvelope()
    #print("*"*20)
    #print(geometry.GetEnvelope())

    linearRing = osgeo.ogr.Geometry(osgeo.ogr.wkbLinearRing)
    linearRing.AddPoint(minEasting, minNorthing)
    linearRing.AddPoint(maxEasting, minNorthing)
    linearRing.AddPoint(maxEasting, maxNorthing)
    linearRing.AddPoint(minEasting, maxNorthing)
    linearRing.AddPoint(minEasting, minNorthing)
    mbr = osgeo.ogr.Geometry(osgeo.ogr.wkbPolygon)
    mbr.AddGeometry(linearRing)
    mbrfeature = osgeo.ogr.Feature(destinationLayer.GetLayerDefn())
    mbrfeature.SetGeometry(mbr)
    mbrfeature.SetField(sourcefieldname, ftrName)
    destinationLayer.CreateFeature(mbrfeature)
    mbrfeature.Destroy()

    feature = sourcelayer.GetNextFeature()

shapefile.Destroy()
```

```
destinationFile.Destroy()  
print ("Datei wurde erstellt: %s" %(destinationlayername))
```

Datenbankverbindung

Als nächstes wird der Datenverkehr von und zu einer räumlichen Datenbank eingeführt. Es handelt sich dabei um eine PostgreSQL Datenbank mit dem Zusatz PostGIS. Damit die in der Datenbank vorhandenen Daten gelesen werden können, müssen folgende Credentials und Parameter bekannt sein:

- Datenbankname
- Hostname
- Anwendername / User
- Passwort

Der Zugang zu den Daten wird mittels des Python-Moduls `psycopg2` hergestellt. Die Methode `connect()`, welche die erwähnten Parameter als Übergabevariablen erwartet, stellt die Verbindung zur Datenbank her. Ist die Verbindung hergestellt worden, können mittels eines so genannten `cursor()` Objekts Abfragen oder Datenmutationen in Form von SQL-Statements (structured query language) abgesetzt werden.

Die Verbindung kann bei zuvor definierten Variablen wie folgt aufgebaut werden:

```
#Connect to PostgreSQL  
connection = psycopg2.connect(dbname=database, host=host, user=usr,  
password=pwd, port="5432")  
cursor = connection.cursor()
```

```
print("connection worked")
```

Zuerst wird aus Sicherheitsgründen eine Methode erstellt, welche die Zugangsdaten aus einer externen Datei einliest.

```
In [ ]: import os  
  
def load_db_credentials(filepath):  
    """  
    Liest eine Datei mit Datenbank-Credentials im Key=Value-Format ein  
    und gibt ein Dictionary mit den Werten zurück.  
  
    :param filepath: Pfad zur Datei mit den Credentials  
    :return: Dictionary mit den Schlüsseln 'host', 'dbname', 'usr', 'pwd'  
    """  
    credentials = {}  
    try:  
        with open(filepath, "r") as file:  
            for line in file:  
                key, value = line.strip().split("=")  
                credentials[key.lower().strip()] = value.strip() # Schlüssel in Kle  
                #print(key,value)  
  
    except FileNotFoundError:  
        print(f"Fehler: Datei '{filepath}' nicht gefunden.")  
    except Exception as e:  
        print(f"Fehler beim Einlesen der Datei: {e}")  
  
    return {  
        "host": credentials.get("host"),  
        "dbname": credentials.get("dbname"),  
        "usr": credentials.get("user"),
```

```
        "pwd": credentials.get("password")
    }
```

```
In [ ]: import os
import psycopg2
import sys
import time
from osgeo import ogr
startTime = time.time()

# Credentials laden
credentials = load_db_credentials("dbcredentials.txt")

database = credentials.get("dbname")
host = credentials.get("host")
usr = credentials.get("usr")
pwd = credentials.get("pwd")

#Connect to PostgreSQL
connection = psycopg2.connect(dbname=database, host=host, user=usr, password=pwd, po
cursor = connection.cursor()

print("connection worked")
```

SQL-Select Statement

Ein einfaches SQL Statement könnte wie folgt aussehen:

```
select * from gemeinden_solothurn;
```

Wird diese Abfrage dem Cursor Statement übergeben, kann anschliessend mit einer Schleife über das Resultat-Set iteriert und die erhaltenen Daten können ausgegeben werden. Dabei kommen folgende Befehle zum Einsatz:

```
cursor.execute(<sqlstring>) und cursor.fetchall()
```

Mit den Befehlen

```
cursor.close() und connection.close()
```

wird die Verbindung zur Datenbank wiederum sauber geschlossen und beendet.

```
In [ ]: sqlstring = "select * from gemeinden_solothurn;"
cursor.execute(sqlstring)

# Spaltennamen abrufen
column_names = [desc[0] for desc in cursor.description]

# Ergebnisse iterieren
for row in cursor.fetchall():
    print("Datensatz:")
    for col_name, value in zip(column_names, row):
        if col_name != 'geom':
            print(f"{col_name}: {value}")
    print("----")

# Verbindung schließen
cursor.close()
connection.close()
```

PostGIS spezifische Befehle

PostGIS bietet eine äusserst umfangreiche Palette an Methoden und Eigenschaften für die SQL basierte Abfrage von spezifisch räumlichen Daten an (vgl. <https://postgis.net/docs/manual-3.4/de/reference.html>)

Aufgabe:

Erstelle eine Abfrage, die für alle Gemeinden des Kantons Solothurn, deren Name mit 'A' beginnt, deren Name, die Gemeindenummer, die Fläche und die Koordinaten der Zentroide ausgibt.

```
In [ ]: #Connect to PostgreSQL
connection = psycopg2.connect(dbname=database, host=host, user=usr, password=pwd, port=port)
cursor = connection.cursor()

sqlstring = "select name, gmde_nr, ST_Area(geom),ST_X(ST_Centroid(geom)),ST_Y(ST_Centroid(geom)) from gemeinden_solothurn where name like 'A%'"
cursor.execute(sqlstring)

for name, gemnr, flaeche, x, y in cursor:
    print ("Gem: %s, Nr: %s, Fläche: %s, Zentroid X: %s / Y: %s" % (name, gemnr, flaeche, x, y))
```

Aufgabe:

Erstelle mittels einer Abfrage, je eine Liste der X- und Y-Koordinaten der Zentroide aller Gemeinden des Kantons Solothurn.

```
In [ ]: sqlstring = "select ST_X(ST_Centroid(geom)),ST_Y(ST_Centroid(geom)) from gemeinden_solothurn"
cursor.execute(sqlstring)

xList = []
yList = []
for x, y in cursor:
    xList.append(x)
    yList.append(y)

print(f"Liste X-Koordinaten:\n{xList}")
print(f"Liste Y-Koordinaten:\n{yList}")
```

Update

Nebst der klassischen `select` Abfrageform können mittels SQL auch Datenmanipulationsbefehle wie bspw. `update` zur Nachführung von Daten abgesetzt werden:

```
sqlstring = "UPDATE mytable SET name = 'XY' WHERE name = 'yx';"
cursor.execute(sqlstring)
```

Dabei ist wichtig, dass im Anschluss an die Ausführung zur Bestätigung ein sog. `commit()` Befehl abgesetzt wird, der vom Datenbankverbindungsobjekt unterstützt wird:

```
connection.commit()
```

Aufgabe:

Ändere den Namen der Gemeinde 'Rohr' auf 'Roor'.

```
In [ ]: sqlstring = "UPDATE gemeinden_solothurn SET name = 'Roor' WHERE name = 'Rohr';"
        cursor.execute(sqlstring)
        connection.commit()
```

Aufgabe:

Es liegt auf der Hand, dass somit auch Daten in die Datenbank selbst eingetragen werden können, dh. dass ein Import von mehreren Datensätzen über Python erfolgt.

Als Übung sollen die Gemeindedaten von Solothurn in die PostgreSQL Datenbank importiert und in der Tabelle `gemso2025` abgelegt werden.

Dazu wird mit dem `Import` SQL Befehl gearbeitet. Zuerst muss ggf. allerdings die notwendige Tabelle in der entsprechenden Datenbank erstellt werden. Dies erfolgt mittels `create Table` Befehl. Mehr dazu kann unter https://www.w3schools.com/sql/sql_create_table.asp nachgelesen werden. Anschliessend sind alle bestehenden Objekte der Gemeinde-Daten aus der Shape-Datei auszulesen, in ein SQL-passendes Format zu übertragen und dann in die neue Tabelle zu importieren.

```
In [ ]: import psycopg2
        import sys
        import time
        from osgeo import ogr
        startTime = time.time()

        # Credentials laden
        credentials = load_db_credentials("dbcredentials.txt")

        database = credentials.get("dbname")
        host = credentials.get("host")
        usr = credentials.get("usr")
        pwd = credentials.get("pwd")
        tabName = 'gemso2025'

        #Connect to PostgreSQL
        connection = psycopg2.connect(dbname=database, host=host, user=usr, password=pwd, p
        cursor = connection.cursor()

        print("connection worked")
        cursor.execute(f"DROP TABLE IF EXISTS {tabName};")
        cursor.execute("drop index if exists gemnrIndex2;")
        cursor.execute("drop index if exists geomIndex2;")

        #Tabelle erstellen inkl. notwendiger Indices
        cursor.execute(f"""CREATE TABLE {tabName} (
                           id          SERIAL,
                           gemnr       INTEGER,
```

```

        beznr          INTEGER,
        name           CHARACTER VARYING(100),

        PRIMARY KEY (id))
    """
cursor.execute(f"CREATE INDEX gemnrIndex2 ON {tabName}(Gemnr)")
cursor.execute(f"SELECT AddGeometryColumn('{tabName}', 'geom', 21781, 'MULTIPOLYGON'
cursor.execute(f"CREATE INDEX geomIndex2 ON {tabName} USING GIST (geom)")
#WICHTIG: speichern der Transaktion!
connection.commit()

#####
#import aller Datensätze der Gemeinden von Solothurn
shapefile = ogr.Open(os.path.join(wpath,"Gemeinden_Solothurn.shp"))
if shapefile is None:
    print ("Datensatz konnte nicht geöffnet werden.\n")
    sys.exit()

layer = shapefile.GetLayer(0)

#Extrahiere Gemeindegeometrie
gemgeometry = None
for feature in layer:
    #Extrahiere Gemeinde-Name
    gemname = feature.GetField("gmde_name")
    print ("Gemeinde %s in Datenbank eingetragen." %gemname)
    #Extrahiere Gemeinde- und Bezirks-Nummer
    gemnr = int(feature.GetField("gmde_nr"))
    beznr = int(feature.GetField("bzrk_nr"))
    #Extrahiere Geometrie Polygon
    gemgeometry = feature.GetGeometryRef()
    gemgeometryaswkt = gemgeometry.ExportToWkt()
    sqlstring = "INSERT INTO gemso2025 (name, gemnr, beznr, geom) VALUES ('%s', %s,
    #print (sqlstring)
    cursor.execute(sqlstring)
    connection.commit()
endTime = time.time()
print ("Took %.4f seconds" % (endTime-startTime))

```

Aufgabe:

Erstelle eine neue Tabelle 'randpoints' mit den Attributen id (Integer, not null, unique), name (string), Bemerkung (string), x (float/double), y (float/double) in LV95 und erstelle n zufällig platzierte Punkte mit fortlaufender ID und zufälligen Namen. Fülle die Koordinatenwerte in die Spalten x und y ab.

Lösung für n=100:

 No description has been provided for this image

```

In [ ]: import psycpg2
import random
import string

# Verbindung zur PostgreSQL-Datenbank herstellen
# Credentials laden
credentials = load_db_credentials("dbcredentials.txt")

```

```

database = credentials.get("dbname")
host = credentials.get("host")
usr = credentials.get("usr")
pwd = credentials.get("pwd")

#Connect to PostgreSQL
connection = psycopg2.connect(dbname=database, host=host, user=usr, password=pwd, port=5432)
cursor = connection.cursor()

print("connection worked")

# Tabelle 'randompoints' erstellen
create_table_query = """
CREATE TABLE IF NOT EXISTS randompoints (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    bemerkung TEXT,
    x DOUBLE PRECISION NOT NULL,
    y DOUBLE PRECISION NOT NULL,
    geom GEOMETRY(Point, 2056) NOT NULL
);
"""

cursor.execute(create_table_query)
connection.commit()

# Funktion zur Generierung zufälliger Namen
def generate_random_name(length=8):
    return ''.join(random.choices(string.ascii_letters, k=length))

# Zufällige Punkte einfügen
def insert_random_points(n):
    insert_query = """
INSERT INTO randompoints (name, bemerkung, x, y, geom)
VALUES (%s, %s, %s, %s, ST_SetSRID(ST_Point(%s, %s), 2056));
"""

    for i in range(n):
        # Zufällige Koordinaten (innerhalb eines Bereichs in der Schweiz, EPSG:2056)
        x = random.uniform(2600000, 2700000) # Zufällige Ost-Koordinate
        y = random.uniform(1200000, 1300000) # Zufällige Nord-Koordinate
        name = generate_random_name()
        bemerkung = "Zufälliger Punkt"

        # Daten einfügen
        cursor.execute(insert_query, (name, bemerkung, x, y, x, y))

    connection.commit()

# Anzahl der zu generierenden Punkte
n = 100 # Beispiel: 10 Punkte
insert_random_points(n)

# Verbindung schließen
cursor.close()
connection.close()

print(f"{n} zufällige Punkte wurden in die Tabelle 'randompoints' eingefügt.")

```

Geowebdienste

Geowebdienste wie WMS und WFS können ebenso über Python aufgerufen und anschliessend die zurückerhaltenen Daten ausgewertet werden. Dazu müssen die entsprechenden URLs

aufgerufen und verarbeitet werden. Im Folgenden werden je ein Beispiel für einen WMS und WFS Aufruf aufgeführt.

WMS Aufruf

Ein Web Map Service (WMS) ist ein standardisierter Webdienst, der es ermöglicht, georeferenzierte Kartenbilder über das Internet abzurufen. Diese Bilder werden dynamisch aus Geodaten generiert und in Formaten wie PNG, JPEG oder GIF bereitgestellt. WMS-Dienste liefern ausschließlich Rasterdaten und eignen sich hervorragend für die Visualisierung von Geoinformationen.

Ein Beispielaufruf eines WMS:

```
http://example.com/wms?  
SERVICE=WMS&VERSION=1.3.0&REQUEST=GetMap&LAYERS=layer_name&STYLES=&CRS=EPSG:4
```

Erklärung der Parameter:

SERVICE=WMS : Gibt an, dass es sich um einen WMS-Dienst handelt.

VERSION=1.3.0 : Spezifiziert die WMS-Version.

REQUEST=GetMap : Fordert eine Karte an.

LAYERS=layer_name : Der Name der abzurufenden Ebene.

STYLES= : Definiert den Stil der Ebene; kann leer bleiben, wenn der Standardstil verwendet wird.

RS=EPSG:4326 : Koordinatenreferenzsystem, hier WGS 84.

BBOX=left,bottom,right,top : Die Begrenzungsbox, die den gewünschten Kartenausschnitt definiert.

WIDTH=800&HEIGHT=600 : Die Abmessungen des zurückgegebenen Bildes in Pixeln.

FORMAT=image/png : Das gewünschte Bildformat.

Als Ergebnis wird bei erfolgreichem Aufruf ein Bild zurückgeliefert, das anschliessend visualisiert oder sonst weiterverarbeitet werden kann.

Der Aufruf in Python kann mit dem Modul `urllib` und dem Befehl

```
urllib.request.urlopen(url)
```

aufgerufen und anschliessend die Antwort verarbeitet werden.

```
In [ ]: # -*- coding: utf-8 -*-  
import os, shutil, sys  
import urllib.request  
from osgeo import gdal  
from osgeo.gdalconst import *  
  
def createWorldFile(geotransform, fileName):  
    # create the 3-band raster file  
    dst_ds = gdal.Open(fileName)  
    dst_ds.SetGeoTransform(geotransform) # specify coords  
    srs = osr.SpatialReference() # establish encoding  
    srs.ImportFromEPSG(2056) # WGS84 lat/long  
    dst_ds.SetProjection(srs.ExportToWkt()) # export coords to file  
    dst_ds.FlushCache() # write to disk  
    dst_ds = None
```



```

def download(url, dest, fileName=None):
    try:
        r= urllib.request.urlopen(url)
        fileName = os.path.join(dest, fileName)
        with open(fileName, 'wb') as f:
            shutil.copyfileobj(r,f)
        r.close()
        print("Successfully downloaded resource {}".format(url))
    except:
        print("ERROR Downloading resource {}".format(url))

path2save2 = wpath #Zielpfad
wmsfile = "wms.tif"
minX = 2500000
maxX = 2600000
minY = 1060000
maxY = 1140000
width = 800
height = 582
pixSize = (maxX-minX)/width
myBB = f"{minX},{minY},{maxX},{maxY}"
geotransform = []
geotransform.append(minX)
geotransform.append(pixSize)
geotransform.append(0.0)
geotransform.append(minY)
geotransform.append(0.0)
geotransform.append(pixSize*-1)

wmslink = f"https://wms.geo.admin.ch/?SERVICE=WMS&REQUEST=GetMap&VERSION=1.3.0&LAYE
download(wmslink,path2save2,wmsfile)
createWorldFile(geotransform,os.path.join(path2save2,wmsfile))

```

Iteration über Punkte und Download der abgeleiteten Kacheln des WMS Dienstes

Aufgabe:

Iteriere über alle Zentroide der Gemeinde und lade für jede Gemeinde eine WMS Kachel herunter.

```

In [ ]: # -*- coding: utf-8 -*-
import os, shutil, sys
import urllib.request
from osgeo import gdal
from osgeo.gdalconst import *

printFlag = False

def handleDirectory(path):
    # Prüfen, ob das Verzeichnis existiert
    if os.path.exists(path):
        print(f"Verzeichnis '{path}' existiert bereits. Lösche es...")
        # Verzeichnis samt Inhalt löschen
        shutil.rmtree(path)
    else:
        print(f"Verzeichnis '{path}' existiert nicht. Erstelle es...")

```

Neues Verzeichnis erstellen

```

os.makedirs(path)
print(f"Verzeichnis '{path}' wurde neu erstellt.")

def download(url, dest, fileName=None, printFlag=True):
    try:
        r = urllib.request.urlopen(url)
        fileName = os.path.join(dest, fileName)
        with open(fileName, 'wb') as f:
            shutil.copyfileobj(r, f)
        r.close()

        if printFlag:
            print("Successfully downloaded resource {}".format(url))
    except Exception as e:
        if printFlag:
            print(e)
            print("ERROR Downloading resource {}".format(url))

path2save2 = os.path.join(wpath, "WMSBulkdownload") #Zielpfad
handleDirectory(path2save2)
for i in range(len(xList)):
    x = xList[i]
    y = yList[i]
    bb = f"{x},{y},{x+5000},{y+5000}"

    wmsfile = f"wms{bb}.gif"
    wmslink = f"https://wms.geo.admin.ch/?SERVICE=WMS&REQUEST=GetMap&VERSION=1.3.0&"
    download(wmslink, path2save2, wmsfile, printFlag=False)

```

WFS Aufruf

Ein Web Feature Service (WFS) hingegen ermöglicht den Zugriff auf die tatsächlichen Geodaten in Vektorform. Mit WFS können Nutzer nicht nur Karten anzeigen, sondern auch die zugrunde liegenden Geometrien und Attribute abrufen, analysieren und bearbeiten. Die Daten werden oft im GML-Format (Geography Markup Language) bereitgestellt.

Beispiel für einen WFS-Aufruf:

```

http://example.com/wfs?
SERVICE=WFS&VERSION=2.0.0&REQUEST=GetFeature&TYPENAMES=feature_type&OUTPUTFORMAT=

```

Erklärung der Parameter:

SERVICE=WFS : Gibt an, dass es sich um einen WFS-Dienst handelt.

VERSION=2.0.0 : Spezifiziert die WFS-Version.

REQUEST=GetFeature : Fordert Geoobjekte an.

TYPENAMES=feature_type : Der Name des abzurufenden Feature-Typs.

OUTPUTFORMAT=application/json : Das gewünschte Ausgabeformat, hier GeoJSON.

Der Hauptunterschied zwischen WMS und WFS liegt darin, dass WMS vorgerenderte Kartenbilder liefert, während WFS direkten Zugriff auf die zugrunde liegenden Vektordaten bietet. WMS ist ideal für die schnelle Visualisierung von Karten, während WFS für Anwendungen geeignet ist, die eine tiefere Interaktion mit den Geodaten erfordern, wie z.B. räumliche Analysen oder Datenbearbeitung.

Für eine detaillierte Anleitung zur Nutzung von WMS und WFS, einschließlich weiterer Beispiele und einer ausführlichen Erklärung der Parameter, kann die folgende Ressource hilfreich sein:
[Anleitung Nutzung WMS und WFS](#)

Während ein WMS Dienst eine Bilddatei als Antwort sendet, liefert ein WFS Dienst bei erfolgreichem Aufruf eine GML Datei mit Geometriedaten und Attributen, also einen Vektordatensatz. Dieser kann im Anschluss weiterverarbeitet werden (bspw. Formatumwandlung etc.).

Das Vorgehen für den Aufruf entspricht im Wesentlichen demjenigen des Aufrufs eines WMS.

```
In [ ]: # -*- coding: utf-8 -*-
import os, sys, shutil
import urllib.request
from osgeo import ogr
from osgeo import gdal

#WFS Daten von:
#http://www.are.zh.ch/internet/audirektion/are/de/geoinformation/geodienste_uebersicht

def download(url, dest, fileName=None):
    #based on:
    #http://stackoverflow.com/questions/862173/how-to-download-a-file-using-python-in-a-shell
    print("*****")
    print("Start downloading of %s" %url)
    print("*****")

    r= urllib.request.urlopen(url)

    try:
        fileName = os.path.join(dest, fileName)
        with open(fileName, 'wb') as f:
            shutil.copyfileobj(r,f)
        print("Saved in %s" %fileName)
    finally:
        r.close()

def convert2shp(path2save2,wfsfile,outputshapefile):
    fn = os.path.join(path2save2,wfsfile)

    driver = ogr.GetDriverByName('ESRI Shapefile')
    if os.path.exists(outputshapefile):
        driver.DeleteDataSource(outputshapefile)

    #convert GMLfile to shape - if needed...
    #ogr2ogrstring = 'ogr2ogr -f "ESRI Shapefile" %s %s' %(outputshapefile,fn)
    #print (ogr2ogrstring)
    #os.system(ogr2ogrstring)
    #print ("Conversion successful...")
    rcmd = ["ogr2ogr", "-f", "ESRI Shapefile", outputshapefile,fn]
    subprocess.run(rcmd)

    #... oder GML...
    wfsfile = ogr.Open(fn)
    if wfsfile is None:
        print("Datensatz konnte nicht geöffnet werden.\n")
        sys.exit( 1 )

    layer = wfsfile.GetLayer(0)
    lname = layer.GetName()
```

```

#Print out number of records:
numftrs = layer.GetFeatureCount()
print ("Anzahl Features in GML Datei: %d" %numftrs)
print ("")

print ("Count Field Count", layer.GetLayerDefn().GetFieldCount())
for feat in range(numftrs):
    for i in range(layer.GetLayerDefn().GetFieldCount()):
        field_defn = layer.GetLayerDefn().GetFieldDefn(i)
    try:
        print (" %s: %s" %(field_defn.GetName(), layer.GetFeature(feat).GetFieldAsText(i)))
    except:
        pass
    print()

#Get Extent
extent = layer.GetExtent()
print ("Ausdehnung:", extent)
print ("Oben-links:", extent[0], extent[3])
print ("Unten-rechts:", extent[1], extent[2])

if __name__=='__main__':

    #####
    #Punktdaten:
    wfsfile = "testpoints.gml"
    path2save2 = wpath
    outputshapefile = os.path.join(path2save2,'wfstestpoints.shp')
    wfsurl = "http://maps.zh.ch/wfs/HaltestellenZHWFS?SERVICE=WFS&VERSION=1.1.0&Request=getFeature"
    download(wfsurl,path2save2 ,wfsfile)

    convert2shp(path2save2,wfsfile,outputshapefile)

    #Linien Daten:
    wfsfile = "testlines.gml"
    path2save2 = wpath
    outputshapefile = os.path.join(path2save2,'wfstestlines.shp')

    wfsurl = "http://maps.zh.ch/wfs/GemZHWFS?SERVICE=WFS&VERSION=1.1.0&Request=getFeature"
    download(wfsurl,path2save2 ,wfsfile)
    convert2shp(path2save2,wfsfile,outputshapefile)

```

[Top](#)

Teil V Weitere Bibliotheken

Shapely

Das Python-Modul Shapely ist eine Bibliothek für die Manipulation und Analyse geometrischer Objekte. Es wird häufig in GIS-Anwendungen und Geodatenanalysen verwendet. Shapely stellt geometrische Typen wie Punkte, Linien und Polygone bereit und ermöglicht die Durchführung von Operationen wie Schnittmengen, Puffer, Union, Differenz und mehr.

Wichtige Features von Shapely

- Geometrische Typen: Punkt, Linie, Polygon, MultiPoint, MultiLineString, MultiPolygon.

- Geometrieoperationen:
 - Schnittmenge: `intersection()`
 - Vereinigung: `union`
 - Differenz: `difference`
 - Symmetrische Differenz: `symmetric_difference`
- Prüfungen:
 - Punkt innerhalb eines Polygons: `contains()`, `within()`
 - Überschneidung: `intersects()`
 - Geometrien berühren sich: `touches()`
 - Maßberechnungen:
 - Abstand zwischen Geometrien: `distance()`
 - Fläche eines Polygons: `area`
 - Umfang eines Polygons: `length`

Es folgt ein Codebeispiel:

```
from shapely.geometry import Point, Polygon

punkt = Point(1, 1)
polygon = Polygon([(0, 0), (2, 0), (2, 2), (0, 2), (0, 0)])

print(f"Punkt innerhalb des Polygons: {polygon.contains(punkt)}")
print(f"Abstand zwischen Punkt und Polygon: {polygon.distance(punkt)}")

puffer = punkt.buffer(1)
print(f"Puffer um den Punkt: {puffer}")
```

Weiterführende Informationen zu Shapely sind zu finden unter [Shapely Homepage](#)

Falls Shapely noch installiert werden muss:

```
!pip install shapely --no-binary shapely
```

Aufgabe:

Gib für jede Gemeinde des Kantons Solothurn deren Zentroid-Koordinaten und die Flächenmasszahl aus.

```
In [ ]: from osgeo import ogr
from shapely import wkt

shapefile = ogr.Open(os.path.join(wpath, "Gemeinden_Solothurn.shp"))
if shapefile is None:
    print ("Datensatz konnte nicht geoeffnet werden.\n")
    sys.exit()

layer = shapefile.GetLayer(0)
```

```

#Gemeindegeometry extrahieren:
geometry = None
cnt = 0
feature = layer.GetNextFeature()
for feature in layer:
    while cnt < 110:
        #Extract Gemeinde-Name
        gemname = feature.GetField("gmde_name")
        #Get Geometry (Polygon)
        gemgeometry = feature.GetGeometryRef()
        #Convert Geometry to shapely-geometry
        gemgeomaswkt = gemgeometry.ExportToWkt()
        shapelypolygon = wkt.loads(gemgeomaswkt)
        #Extract Centroid
        centroid_point = shapelypolygon.centroid
        x=centroid_point.x
        y=centroid_point.y
        area = shapelypolygon.area
        #Printout Information
        print ("Gemeinde %s hat folgenden Zentroid: (%f, %f) und folgende Flaeche %f" % (gemname, centroid_point.x, centroid_point.y, area))
        cnt += 1
        feature = layer.GetNextFeature()

```

Aufgabe:

Gib die Stützpunkte des gemeinsamen Grenzverlaufs zweier benachbarten Gemeinden des Kantons Solothurn aus.

```

In [ ]: # Beispiel für die Ermittlung eines gemeinsamen Grenzverlaufs

import osgeo.ogr
import shapely.wkt
import sys

from shapely.geometry import LineString

shapefile = osgeo.ogr.Open(os.path.join(wpath,"Gemeinden_Solothurn.shp"))
if shapefile is None:
    print ("Datensatz konnte nicht geoeffnet werden.\n")
    sys.exit( 1 )

layer = shapefile.GetLayer(0)
#feature = layer.GetFeature(1)

#Gemeindegeometry extrahieren:
geometry = None
for feature in layer:
    #Extract Geometries for Seewen and Nunningen
    if feature.GetField("NAME") == 'Seewen':
        geomSeewen = feature.GetGeometryRef()
        Seewengeomaswkt = geomSeewen.ExportToWkt()
        shapelypolygonSeewen = shapely.wkt.loads(Seewengeomaswkt)
    elif feature.GetField("NAME") == 'Nunningen':
        geomNunningen = feature.GetGeometryRef()
        Nunningengeomaswkt = geomNunningen.ExportToWkt()
        shapelypolygonNunningen = shapely.wkt.loads(Nunningengeomaswkt)

#compute intersection

```

```

intersectionline = shapelypolygonSeewen.intersection(shapelypolygonNunningen)

type = intersectionline.geom_type
vertices = len(intersectionline.geoms)
print (""")
print ("Laenge des gemeinsamen Grenzverlaufs (vom Typ %s): %fm mit %i Liniensegmente")
print (""")

#Extraktion of Vertices of intersectionline
i=0
for vertex in intersectionline.geoms:
    i=i+1
    x1 = vertex.coords[0][0] # 1. Punkt der Linie, X-Koordinate
    y1 = vertex.coords[0][1] # 1. Punkt der Linie, Y-Koordinate

    print ("Stuetzpunkt[%i]: (x=%f, y=%f)" % (i,x1,y1))
    if i==len(intersectionline.geoms):
        x1=vertex.coords[1][0] # 2. Punkt der Linie, X-Koordinate
        y1=vertex.coords[1][1] # 2. Punkt der Linie, Y-Koordinate
        print ("Stuetzpunkt[%i]: (x=%f, y=%f)" % (i+1,x1,y1))

```

Fiona

Das Python-Modul Fiona ist eine Bibliothek zur Arbeit mit Geodatenformaten wie Shapefiles, GeoJSON und anderen vektor-basierten Geodatenformaten. Es baut auf der Geodatenbibliothek **GDAL/OGR** auf und bietet eine benutzerfreundliche API für das Lesen, Schreiben und Verarbeiten von Geodaten.

Fiona wird häufig in Kombination mit anderen GIS-Paketen wie **Shapely**, **Geopandas** oder **Rasterio** verwendet.

Wichtige Features von Fiona

- **Lesen von Geodaten:** Öffne vektorbasierte Dateien und iteriere über deren Features.
- **Schreiben von Geodaten:** Erstelle neue Geodateien mit benutzerdefinierten Attributen und Geometrien.
- **Unterstützte Formate:** Shapefile, GeoJSON, GPKG, und viele andere durch GDAL unterstützte Formate.
- **Kontextmanager:** Nutzt Python's **with-Statement** für sicheres Datei-Handling.
- **Schema-Definitionen:** Definiere die Struktur und Attribute der Daten.

Code-Beispiel

Lesen von Geodaten:

```

import fiona

with fiona.open("gemeinden_solothurn.shp", "r") as source:
    # Metadaten der Datei anzeigen
    print("Schema:", source.schema)
    print("CRS (Koordinatensystem):", source.crs)

    for feature in source:
        print("Feature ID:", feature['id'])
        print("Geometrie:", feature['geometry'])
        print("Attribute:", feature['properties'])

```

Schreiben von Geodaten:

```
import fiona

from shapely.geometry import mapping, Point

schema = {
    'geometry': 'Point',
    'properties': {'name': 'str'},
}

with fiona.open(
    "punkte.geojson", "w",
    driver="GeoJSON",
    schema=schema,
    crs="EPSG:4326"
) as sink:
    punkt = Point(7.5, 47.1)
    sink.write({
        'geometry': mapping(punkt),
        'properties': {'name': 'Beispielpunkt'},
    })
```

Weiterführende Informationen zu Fiona sind zu finden unter [Weitere Infos zu Fiona](#)

Ausgabe der Basisinformationen mit Fiona:

```
fiona.open()
```

```
In [ ]: import fiona

c = fiona.open(os.path.join(wpath, 'Gemeinden_Solothurn.shp'), 'r')
print("Anzahl Datensätze: %i " %len(list(c)))
print("Format: %s" %c.driver)
print("Geo-Referenzsystem: %s" %c.crs)
print("Ausdehnung: %s" %str(c.bounds))
```

Aufgabe:

Erstelle einen neuen Punktlayer mit Fiona, der n zufällig räumlich verteilte Punkte enthält in einem bestimmten durch einen Koordinatenwertebereich definierten Gebiet.

```
In [ ]: import fiona
# Shapely wird für die Definition der Geometrie benötigt
from shapely.geometry import Point, LineString, Polygon, mapping
import random
#from pathvariable import datapath

npnts = 400

# Schema: einfaches Dictionary mit Geometrie und Properties als keys
```



```

schema_pnt = {'geometry': 'Point', 'properties': {'Id': 'int', 'Name': 'str'}}

# Ein paar Punktgeometrien
points = [Point(272830.63, 155125.73), Point(273770.32, 155467.75), Point(273536.47, 155467.75)]
expFl = os.path.join(wpath, 'myrandompointshapes2.shp')
with fiona.open(expFl, 'w', 'ESRI Shapefile', schema_pnt) as pntlayer:
    for cnt in range(1, npnts):
        # Schema befüllen
        elem = {}
        # Geometrie wird mit der mapping function von shapely erstellt
        y=random.randrange(1000000, 2000000)
        x=random.randrange(2000000, 3000000)
        pnt = Point(x,y)
        elem['geometry'] = mapping(pnt)
        # Attributwerte
        elem['properties'] = {'Name': 'Punkt ' + str(cnt), 'Id' : cnt}
        # Erstellen des neuen Datensatzes / Records
        pntlayer.write(elem)

```

Folium

Folium ist eine Python-Bibliothek zur Erstellung interaktiver Karten mit Leaflet.js, einer führenden Open-Source-Bibliothek für webbasierte Kartenvisualisierung. Mit Folium kannst du schnell und einfach interaktive Karten erstellen und Daten wie Marker, Polygone oder Heatmaps visualisieren.

Wichtige Features von Folium

- **Interaktive Karten:** Erstellung von Karten mit Zoom, Schwenk und Layer-Steuerung.
- **Verschiedene Basiskarten:** Unterstützung für Karten von OpenStreetMap, Stamen und anderen Kartenanbietern.
- **Datenvisualisierung:** Darstellung von Geodaten in verschiedenen Formaten wie GeoJSON, Shapefiles (indirekt), und Pandas-DataFrames.
- **Erweiterte Layer:** Heatmaps, Choroplethen, Markercluster, Pop-ups und vieles mehr.
- **Export:** Speichere die erstellten Karten als HTML-Dateien zur einfachen Weitergabe oder zum Hosten.

Weitere Informationen zu Folium sind zu finden unter [Weitere Infos zu Folium/a>](#)

Code-Beispiel

Einfache Karte erstellen

```
import folium
```

```
#Karte mit einem Startpunkt erstellen
```

```
karte = folium.Map(location=[47.1, 7.5], zoom_start=12)
```

```
#Karte als HTML speichern
```

```
karte.save("simple_map.html")
```

Marker hinzufügen

```
import folium
```

```
# Karte erstellen
```

```
karte = folium.Map(location=[47.1, 7.5], zoom_start=12)
```

```
# Marker hinzufügen
```

```
folium.Marker([47.1, 7.5], popup="Solothurn", tooltip="Klicken für Info").add_to(karte)
```

```
folium.Marker([47.2, 7.6], popup="Zuchwil", tooltip="Weitere Info",  
icon=folium.Icon(color="green")).add_to(karte)
```

```
# Karte speichern
```

```
karte.save("map_with_markers.html")
```

GeoJSON-Daten visualisieren

```
import folium
```

```
# Karte erstellen
```

```
karte = folium.Map(location=[47.1, 7.5], zoom_start=10)
```

```
# GeoJSON hinzufügen
```

```
geojson_url = "https://raw.githubusercontent.com/python-  
visualization/folium/main/examples/data/world-countries.json"  
folium.GeoJson(geojson_url, name="Ländergrenzen").add_to(karte)
```

```
# Layer-Kontrolle hinzufügen
```

```
folium.LayerControl().add_to(karte)
```

```
# Karte speichern
```

```
karte.save("map_with_geojson.html")
```

Choroplethen-Karte

```
import folium
```

```
# Karte erstellen
```

```
karte = folium.Map(location=[47.1, 7.5], zoom_start=8)
```

```
# Choroplethen hinzufügen
```

```
folium.Choropleth(  
    geo_data=geojson_url, # GeoJSON-Daten  
    data={"CHE": 50, "DEU": 100, "FRA": 80}, # Beispiel-Daten  
    columns=["Country", "Value"], # Spalten definieren  
    key_on="feature.id", # Verknüpfungsschlüssel in GeoJSON  
    fill_color="YlOrRd", # Farbpalette  
    fill_opacity=0.7,  
    line_opacity=0.2,  
    legend_name="Beispiel-Werte"  
).add_to(karte)
```

```
# Karte speichern
```

```
karte.save("choropleth_map.html")
```

Aufgabe:

Erstelle eine Karte mit Folium, welche die Gemeinden des Kantons Solothurn zeigt.

```
In [ ]: import folium, json  
m = folium.Map(location=[47.3, 7.61], zoom_start=10)  
  
rfile = open(os.path.join(wpath, 'Gemeinden_SolothurnWGS84.json'), 'r', encoding='utf-8')  
jsonData = json.loads(rfile)
```

```

style_function = {
    'fillColor': 'white',
}
folium.GeoJson(jsonData, name='json_data',#,
               #style_function=lambda x: style_function
               ).add_to(m)

```

m

Aufgabe:

Erstelle eine Karte mit Folium, welche den Standort der ETH Zürich, Höggerberg zeigt.

```

In [ ]: import folium, json
m = folium.Map(location=[47.409, 8.51], zoom_start=10)

infoHtmlText = """<table style="width:100%">
    <tr>
        <th>Company</th>
        <th>Contact</th>
        <th>Country</th>
    </tr>
    <tr>
        <td>Alfreds Futterkiste</td>
        <td>Maria Anders</td>
        <td>Germany</td>
    </tr>
    <tr>
        <td>Centro comercial Moctezuma</td>
        <td>Francisco Chang</td>
        <td>Mexico</td>
    </tr>
</table>"""

folium.Marker(
    location=[47.40875, 8.50778],
    popup=infoHtmlText,
    icon=folium.Icon(color="red", icon="info-sign"),
).add_to(m)

```

m

Leafmap

Leafmap ist eine Python-Bibliothek, die auf Folium, WhiteboxTools, und anderen GIS-Tools aufbaut. Sie ist speziell für interaktive Kartenerstellung, Analyse und Visualisierung von Geodaten optimiert. Leafmap bietet umfangreichere Funktionen für die Arbeit mit Geodaten als Folium, einschließlich der Unterstützung für erweiterte Datenquellen wie GeoJSON, Shapefiles, Rasterdaten und Cloud-Dienste.

Wichtige Features von Leafmap

- **Erstellung interaktiver Karten:** Leafmap basiert auf Leaflet.js und ipywidgets, wodurch interaktive und anpassbare Karten erstellt werden können.

- **Unterstützung von Cloud-Daten:** Arbeiten mit Cloud-Diensten wie Google Earth Engine, OpenStreetMap und anderen APIs.
- **Erweiterte Geodatenformate:** Unterstützung für GeoJSON, Shapefiles, KML, NetCDF, GeoTIFF und Rasterdaten.
- **Analyse-Tools:** Integration von Werkzeugen wie WhiteboxTools für erweiterte Analysen.
- **Choroplethen, Heatmaps und Layer:** Datenvisualisierung in verschiedenen Formen.
- **Integration mit Jupyter Notebooks:** Ideal für die Entwicklung von Karten in interaktiven Python-Umgebungen.

Weiterführende Informationen

Die offizielle Website von Leafmap sowie die Dokumentation sind hier zu finden:

- **Leafmap auf GitHub:** <https://github.com/opengeos/leafmap>
- **Leafmap-Dokumentation:** <https://leafmap.org/>

Dort gibt es auch Tutorials, erweiterte Beispiele und Informationen über unterstützte Datenformate und Funktionen.

Code-Beispiel

Einfache Karte mit Basiskarte

```
import leafmap

# Karte mit Standardansicht erstellen
karte = leafmap.Map(center=[47.1, 7.5], zoom=12)
```

```
# Karte anzeigen
karte
```

GeoJSON hinzufügen

```
import leafmap

# Karte erstellen
karte = leafmap.Map(center=[47.1, 7.5], zoom=10)

# GeoJSON hinzufügen
geojson_url = "https://raw.githubusercontent.com/python-visualization/folium/main/examples/data/world-countries.json"
karte.add_geojson(geojson_url, layer_name="Ländergrenzen")
```

```
# Karte anzeigen
karte
```

Shapefile anzeigen

```
import leafmap

# Karte erstellen
karte = leafmap.Map(center=[47.1, 7.5], zoom=10)

# Shapefile laden
shp_path = "Gemeinden_solothurn.shp"
karte.add_shapefile(shp_path, layer_name="Gemeinden Solothurn")
```

```
# Karte anzeigen
karte
```

Choroplethen-Karte

```
import leafmap
```

```

karte = leafmap.Map(center=[47.1, 7.5], zoom=8)

# Beispiel-Daten
data = {"CHE": 50, "DEU": 100, "FRA": 80}

# Choroplethen erstellen
karte.add_choropleth(
    geo_data=geojson_url,
    data=data,
    key="id", # Verknüpfungsschlüssel in GeoJSON
    value="value", # Spaltenname in den Daten
    legend_title="Beispielwerte"
)

# Karte anzeigen
karte

```

Aufgabe:

Erstelle eine Karte mit Leafmap, welche die Gemeinden des Kantons Solothurn zeigt und den WMS der Swisstopo PK25 farbig als Hintergrundkarte verwendet.

```

In [ ]: import leafmap

# Karte erstellen
karte = leafmap.Map(center=[47.250, 7.65], zoom=10)

# GeoJSON hinzufügen
geojson= os.path.join(wpath,"Gemeinden_SolothurnWGS84.json")
karte.add_geojson(geojson, layer_name="Gemeinden Solothurn")

pk25 = "https://wms.geo.admin.ch/?"
karte.add_wms_layer(
    url=pk25,
    layers="ch.swisstopo.pixelkarte-farbe-pk25.noscale",
    name="PK25",
    #attribution="MRLC",
    format="image/png",
    shown=True,
)

# Karte anzeigen
karte

```

Aufgabe:

Erstelle eine Karte mit Leafmap, welche als Dichtekarte (Heatmap) die Bevölkerungsdichte weltweit zeigt.

Hinweis: Die Quelle der Daten ist

https://raw.githubusercontent.com/opengeos/leafmap/master/examples/data/world_cities.csv

```
In [ ]: # add a heatmap
m2 = leafmap.Map()

in_csv = "https://raw.githubusercontent.com/opengeos/leafmap/master/examples/data/v
m2.add_heatmap(
    in_csv,
    latitude="latitude",
    longitude="longitude",
    value="pop_max",
    name="Heat map",
    radius=20,
)
colors = ["blue", "lime", "red"]
vmin = 0
vmax = 10000
m2.add_colorbar(colors=colors, vmin=vmin, vmax=vmax)
m2.add_title("World Population Heat Map", font_size="20px", align="center")
m2
```

DuckDB

DuckDB ist eine schnelle, spaltenbasierte, analytische SQL-Datenbank, die speziell für lokale Analysen entwickelt wurde. Sie ist als In-Memory-Datenbank optimiert, kann aber auch große Datenmengen auf Festplatten effizient verarbeiten. DuckDB eignet sich besonders für Datenanalysen, Machine Learning Workflows und Embedded-Analytics.

Wichtige Features von DuckDB

- **Blitzschnelle Performance** – Optimierte für analytische Abfragen (OLAP) auf großen Datenmengen.
- **Einfache Installation & Nutzung** – Kein separater Server erforderlich, läuft als Embedded-Datenbank.
- **Unterstützt viele Formate** – Kann mit CSV, Parquet, JSON, SQLite, Pandas DataFrames, Arrow und mehr arbeiten.
- **Standard-SQL-Unterstützung** – DuckDB verwendet ANSI-SQL mit leistungsstarken Erweiterungen.
- **In-Memory- & Festplatten-Optimierung** – DuckDB kann sowohl im Speicher als auch auf Festplatte betrieben werden.
- **Multi-Threading** – Nutzt mehrere CPU-Kerne für parallele Abfrageverarbeitung.

Code-Beispiel

DuckDB als In-Memory-Datenbank verwenden

```
import duckdb

# Verbindung zur In-Memory-Datenbank herstellen
con = duckdb.connect(database=':memory:')

# Eine Tabelle erstellen und Daten einfügen
con.execute("CREATE TABLE test (id INTEGER, name TEXT, value FLOAT)")
con.execute("INSERT INTO test VALUES (1, 'A', 10.5), (2, 'B', 20.3), (3, 'C', 15.8)")

# Abfrage ausführen
result = con.execute("SELECT * FROM test").fetchall()
print(result)
```

DuckDB mit einer CSV-Datei verwenden

```
import duckdb
```

```
# Verbindung zur Datenbank
```

```
con = duckdb.connect(database=':memory:')
```

```
# CSV-Datei direkt als virtuelle Tabelle nutzen
```

```
df = con.execute("SELECT * FROM read_csv_auto('data.csv')").df()
```

```
print(df)
```

DuckDB mit Pandas DataFrame nutzen

```
import duckdb
```

```
import pandas as pd
```

```
# Beispiel-DataFrame
```

```
df = pd.DataFrame({'id': [1, 2, 3], 'value': [10.5, 20.3, 15.8]})
```

```
# Verbindung zu DuckDB und DataFrame als Tabelle verwenden
```

```
con = duckdb.connect()
```

```
con.register('df_table', df)
```

```
# SQL-Abfrage direkt auf Pandas DataFrame ausführen
```

```
result = con.execute("SELECT * FROM df_table WHERE value > 15").df()
```

```
print(result)
```

DuckDB mit Parquet-Dateien nutzen

```
import duckdb
```

```
# Parquet-Datei direkt abfragen
```

```
df = duckdb.query("SELECT * FROM 'data.parquet' WHERE value > 100").df()
```

```
print(df)
```

Vergleich: DuckDB vs. SQLite vs. Pandas

Feature	DuckDB	SQLite	Pandas
SQL-Unterstützung	✅ Vollständiges SQL	✅ Vollständiges SQL	❌ Nur DataFrame-APIs
Optimierung für OLAP	✅ Ja (Spaltenbasiert)	❌ Nein (Zeilenbasiert)	✅ Ja, aber langsamer
Multi-Threading	✅ Ja	❌ Nein	❌ Nein
Datenformate	CSV, Parquet, Pandas, Arrow, JSON	Nur CSV	CSV, Excel, JSON, SQL
Performance	🚀 Sehr schnell für große Daten	🐢 Langsam für Analysen	🐢 Langsam bei sehr großen Daten
Server nötig?	❌ Nein	❌ Nein	❌ Nein

Fazit: DuckDB ist eine gute Alternative zu SQLite und Pandas, wenn es um schnelle, analytische Abfragen mit SQL geht!

Weiterführende Informationen

- **Offizielle Website:** <https://duckdb.org/>
- **GitHub Repository:** <https://github.com/duckdb/duckdb>
- **Dokumentation:** <https://duckdb.org/docs/>

DuckDB Spatial Extension (`spatial`)

DuckDB bietet mit `spatial` eine Erweiterung für **geographische und räumliche Datenverarbeitung** an. Diese Erweiterung ermöglicht die Arbeit mit **Geo-Daten** direkt in DuckDB, ähnlich wie PostGIS für PostgreSQL.

Installation der `spatial` -Erweiterung in DuckDB

Die `spatial` -Erweiterung kann einfach mit folgendem Befehl in DuckDB installiert und geladen werden:

```
import duckdb
```

```
# Installation der Spatial Extension
duckdb.install_extension('spatial')
```

```
# Laden der Spatial Extension
duckdb.load_extension('spatial')
```

💡 **Hinweis:**

- Die Installation muss nur einmal erfolgen.
 - Die Erweiterung muss jedoch in jeder neuen Sitzung mit `load_extension('spatial')` aktiviert werden.
-

Funktionen der `spatial` -Erweiterung

Nachdem die Erweiterung geladen wurde, stehen viele **GIS-Funktionen** zur Verfügung, ähnlich wie in PostGIS. DuckDB nutzt dabei **GEOS** und **PROJ**, zwei bekannte GIS-Bibliotheken.

Beispiel-Funktionen:

✅ **Geometrie-Erstellung:** `ST_GeomFromText()`, `ST_Point()`, `ST_Polygon()`, `ST_LineString()`

✅ **Berechnungen:** `ST_Area()`, `ST_Length()`, `ST_Distance()`

✅ **Transformationen:** `ST_Transform()`, `ST_SetSRID()`

✅ **Geometrie-Operationen:** `ST_Union()`, `ST_Intersection()`, `ST_Buffer()`

Beispiele für die Verwendung der Spatial Extension in DuckDB

1 Punkt erstellen und anzeigen

```
import duckdb
```

```
duckdb.load_extension('spatial')
```

```
result = duckdb.query("SELECT ST_AsText(ST_Point(7.5, 47.0))")
print(result.fetchall())
```

📌 **Ergebnis:**

```
[('POINT(7.5 47.0)',)]
```


2 Entfernung zwischen zwei Punkten berechnen

```
result = duckdb.query("""
    SELECT ST_Distance(
        ST_Point(7.5, 47.0),
        ST_Point(7.6, 47.1)
    )
""")
print(result.fetchall())
```

📌 ****Ergebnis:****

```
[(0.14142135623731025,)]
```

3 Fläche eines Polygons berechnen

```
```python
result = duckdb.query("""
 SELECT ST_Area(ST_GeomFromText('POLYGON((0 0, 4 0, 4 3, 0 3, 0
0))'))
""")
print(result.fetchall())
```

📌 **Ergebnis:**

```
[(12.0,)]
```

## 4 Koordinaten transformieren (z.B. von EPSG:4326 nach EPSG:3857)

```
result = duckdb.query("""
 SELECT ST_AsText(
 ST_Transform(
 ST_SetSRID(ST_Point(7.5, 47.0), 4326),
 3857
)
)
""")
print(result.fetchall())
```

📌 **Ergebnis:**

```
[('POINT(834569.45 5921509.66)',)]
```

✓ Dies zeigt die transformierten Koordinaten im **Mercator-Projektionssystem**.

## Vergleich: DuckDB mit Spatial vs. PostGIS

Feature	DuckDB ( spatial )	PostGIS (PostgreSQL)
Installation	Einfach mit <code>install_extension('spatial')</code>	PostgreSQL + PostGIS-Plugin erforderlich
SQL- Unterstützung	✓ SQL-Funktionen wie <code>ST_Point()</code> , <code>ST_Distance()</code>	✓ Vollständige GIS-SQL- Unterstützung
Performance	🚀 Sehr schnell für lokale Analysen	🏗️ Leistungsstark, aber schwergewichtiger

Feature	DuckDB ( spatial )	PostGIS (PostgreSQL)
Server nötig?	❌ Nein (läuft als Embedded DB)	✅ Ja (benötigt PostgreSQL-Server)
Nutzung mit Pandas	✅ Direkt mit Pandas nutzbar	🔄 Extra Setup mit GeoPandas nötig

## Fazit: Wann sollte man DuckDB Spatial verwenden?

- ✅ Ideal für schnelle GIS-Analysen in Python, Jupyter Notebooks oder Pandas
- ✅ Kein Server erforderlich – einfacher als PostGIS
- ✅ Parquet, CSV und Pandas-Support für einfache Datenanalyse
- ❌ Nicht für große Multi-User-Datenbanken geeignet (dann ist PostGIS besser)

## Weiterführende Links

- 🔗 **Offizielle DuckDB Spatial-Dokumentation:**  
<https://duckdb.org/docs/extensions/spatial.html>
- 🔗 **PostGIS als Alternative:**  
<https://postgis.net/>

🚀 **Perfekt für lokale GIS-Analysen!**

```
In []: import duckdb

Erweiterung installieren
duckdb.install_extension('spatial')

Erweiterung laden
duckdb.load_extension('spatial')
```

```
In []: result = duckdb.query("""
 SELECT ST_Distance(
 ST_Point(7.5, 47.0),
 ST_Point(7.6, 47.1)
)
 """)
print(result.fetchall())
```

## Aufgabe:

Importiere die Exceldatei mit allen Hauptstädten Europas "Hauptstaedte\_Europa.xlsx".

- Gib die geladenen Daten in einer Tabellenansicht aus.
  - Erstelle eine Distanzmatrix von jeder Hauptstadt zu einer anderen ausgibt.
- Hinweis: Ggf sind weitere Module wie geopy und openpyxl zu installieren mittels
- ```
!pip install geopy openpyxl!
```

```
In [ ]: !pip install geopy openpyxl
```

```
In [ ]: import duckdb
import pandas as pd
from geopy.distance import geodesic
import geopy

# Excel-Datei laden
capitalsFl = os.path.join(wpath, "Hauptstaedte_Europa.xlsx")
df = pd.read_excel(capitalsFl)

# DuckDB-Verbindung erstellen
db = duckdb.connect()

db.execute("""
    CREATE TABLE hauptstaedte (
        Stadt STRING,
        Land STRING,
        Breitengrad DOUBLE,
        Laengengrad DOUBLE
    )
""")

# Daten einfügen
db.execute("INSERT INTO hauptstaedte SELECT * FROM df")

# Lösche die Tabelle capitals falls sie schon existiert
duckdb.sql("drop table if exists capitals;")

# Lese Daten aus einer Excel-Datei
sqlStmt = f"CREATE TABLE capitals AS SELECT * FROM st_read('{capitalsFl}');"
#runSql(duckdb,sqlStmt)
duckdb.sql(sqlStmt)
# Alle Daten anzeigen
print("Alle Daten in der Tabelle:")
print(duckdb.sql("SELECT * FROM capitals"))
#print(db.execute("SELECT * FROM capitals").fetchdf())

# Distanzmatrix berechnen
hauptstaedte = duckdb.sql("SELECT Stadt, Breitengrad, Längengrad FROM capitals;").fetchdf()
distanzmatrix = []

for city1 in hauptstaedte:
    row = []
    for city2 in hauptstaedte:
        dist = geodesic((city1[1], city1[2]), (city2[1], city2[2])).km
        row.append(dist)
    distanzmatrix.append(row)

# Distanzmatrix als DataFrame
city_names = [city[0] for city in hauptstaedte]
df_distanz = pd.DataFrame(distanzmatrix, index=city_names, columns=city_names)
print("Distanzmatrix in km:")
print(df_distanz)
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]: import nbformat
```

```

# Original-Notebook-Pfad
# Original- und Kopienamen definieren
original_notebook = "gp2025ETHZH.ipynb" # Ersetze mit deinem Original-Notebook
kopie_notebook = "gp2025ETHZH_student.ipynb" # Name der bearbeiteten Kopie

# Erstelle eine Kopie des Notebooks
shutil.copy(original_notebook, kopie_notebook)

# Lade die Kopie
with open(kopie_notebook, "r", encoding="utf-8") as f:
    notebook = nbformat.read(f, as_version=4)

# Neue Liste für Zellen mit eingefügten Markdown-Zellen und leeren Code-Zellen
new_cells = []

for cell in notebook.cells:
    if cell.cell_type == "code":
        # Markdown-Zelle mit Styling für orangenen Hintergrund
        markdown_cell = nbformat.v4.new_markdown_cell(
            '<div style="background-color: orange; padding: 10px; font-weight: bold'
        )
        new_cells.append(markdown_cell)

        # Ersetze die Code-Zelle durch eine leere Code-Zelle
        empty_code_cell = nbformat.v4.new_code_cell("")
        new_cells.append(empty_code_cell)
    else:
        # Behalte alle anderen Zellen unverändert
        new_cells.append(cell)

# Ersetze die Zellen im Notebook
notebook.cells = new_cells

# Speichere die modifizierte Kopie
with open(kopie_notebook, "w", encoding="utf-8") as f:
    nbformat.write(notebook, f)

print(f"Die modifizierte Kopie wurde erstellt: {kopie_notebook}")

```

```

In [ ]: # Erstelle ein neues Notebook nur mit den Markdown-Zellen / ohne Code

import nbformat

# Notebook laden
with open("gp2025ETHZH.ipynb", "r", encoding="utf-8") as f:
    notebook = nbformat.read(f, as_version=4)

# Nur Markdown-Zellen behalten
markdown_cells = [cell for cell in notebook["cells"] if cell["cell_type"] == "markdown"]

# Neues Notebook mit nur Markdown-Zellen erstellen
new_notebook = nbformat.v4.new_notebook()
new_notebook["cells"] = markdown_cells

# Neues Notebook speichern
mdo = "gp2025ETHZH_md_only.ipynb"
with open(mdo, "w", encoding="utf-8") as f:
    nbformat.write(new_notebook, f)

print(f"Neues Notebook mit nur Markdown-Zellen wurde gespeichert als '{mdo}'")

```